東北農業大學

本科毕业论文

基于国产 FPGA 的 RISC-V 软核移植与 FreeRTOS 实现

RISC-V Soft Core Transplantation Based on Domestic FPGA and FreeRTOS Implementation

所在学院：电气与信息学院

所在班级：物联 2102 班

学生姓名：睢红鑫

学生学号：A05210206

指导教师：张喜海 王洁

研究方向：物联网硬件开发

2025 年 06 月

東北農業大學

本科毕业论文

基于国产 FPGA 的 RISC-V 软核移植与 FreeRTOS 实现

RISC-V Soft Core Transplantation Based on Domestic FPGA and FreeRTOS Implementation

所在学院：电气与信息学院

所在班级：物联 2102 班

学生姓名：睢红鑫

学生学号：A05210206

指导教师：张喜海 王洁

研究方向：物联网硬件开发

2025 年 06 月

摘要

随着 RISC-V 指令集架构走向开源开放，基于 RISC-V 的软核处理器于 FPGA 平台上的实现渐渐成为自主芯片设计、嵌入式开发以及物联网终端应用的关键方向，FPGA 拥有灵活可编程、开发周期短以及硬件资源可定制等优点，颇为适合用于软核处理器的快速验证与系统集成。基于 RISC-V 开发的微处理器在综合性能方面竞争力颇强，还可以免受指令集相关专利的限制，为物联网设备开发者给予了开放友好的平台支持。

本文依托国产紫光同创 PGL22G 系列 FPGA 平台，达成了 tinyriscv 软核处理器的适配与移植，构建起包含处理器内核、存储以及外设模块的系统级集成方案。系统运用自定义 RIB 总线架构，达成了软硬件资源的互联，还提升了模块扩展性与可移植性，架构设计层面，处理器核心采用三级流水线结构，兼顾了系统性能与 FPGA 资源利用率，并且依照嵌入式设备的要求完成了外设模块的设计，借助功能仿真与时序验证，保证逻辑正确性和接口规范性。另外系统依靠 CoreMark 基准测试，验证了软核架构的稳定性和运行效率。

在硬件模块搭建完毕后，借助 RISC-V 交叉编译工具链，结合 ModelSim 仿真、波形分析和板级测试，全面验证了系统的功能正确性，同时本文完成了 FreeRTOS 操作系统的移植，验证了在该平台下的多任务调度和中断处理机制，保证 tinyriscv 软核有良好的实时性和稳定性。结果显示，国产 FPGA 平台上可稳定运行 tinyriscv 软核，且有良好的可扩展性和软硬件协同开发潜力。

关键词：RISC-V 指令集；FPGA；微控制器；操作系统移植

RISC-V Soft Core Transplantation Based on Domestic FPGA and FreeRTOS Implementation

Abstract

With the RISC-V instruction set architecture moving towards open source and openness, the implementation of RISC-V soft-core processors on the FPGA platform has gradually become a key direction for independent chip design, embedded development, and Internet of Things terminal applications. FPGA has the advantages of flexible programmability, short development cycle, and customizable hardware resources. It is quite suitable for the rapid verification and system integration of soft-core processors. Microprocessors developed based on RISC-V are highly competitive in terms of comprehensive performance and can also be free from the restrictions of patents related to instruction sets, providing open and friendly platform support for developers of Internet of Things devices.

Based on the domestic Unisplendour Tongchuang PGL22G series FPGA platform, this paper has achieved the adaptation and transplantation of the TinyRISC-V soft-core processor, and constructed a system-level integration solution including processor cores, storage, and peripheral modules. The system adopts a custom RIB bus architecture, achieving the interconnection of software and hardware resources, and also enhancing the scalability and portability of the module. At the architectural design level, the processor core adopts a three-stage pipeline structure, taking into account both system performance and the utilization rate of FPGA resources. Moreover, the design of peripheral modules is completed in accordance with the requirements of embedded devices. With the help of functional simulation and timing verification, Ensure logical correctness and interface standardization. In addition, the system relies on CoreMark benchmark tests to verify the stability and operational efficiency of the soft-core architecture.

After the hardware module was built, with the help of the RISC-V cross-compilation toolchain, combined with ModelSim simulation, waveform analysis and board level testing, the functional correctness of the system was comprehensively verified. Meanwhile, this paper completed the transplantation of the FreeRTOS operating system and verified the multi-task scheduling and interrupt handling mechanism under this platform. Ensure that the TinyRISC-V soft core has good real-time performance and stability. The results show that the TinyRISC-V soft core can run stably on the domestic FPGA platform, and it has good scalability and potential for the collaborative development of software and hardware.

Key words: RISC-V Instruction Set; FPGA; Microcontroller; Operating System Porting

目录

摘要 I
Abstract II
1 前言 1
1.1 研究背景及意义 1
1.2 国内外研究现状 2
1.2.1 国外研究现状 2
1.2.2 国内研究现状 2
1.2.3 发展趋势 3
1.3 主要研究内容与工作 3
2 相关技术背景 4
2.1 指令集架构介绍 4
2.1.1 发展概述 4
2.1.2 RISC-V 指令集架构 5
2.2 FPGA 概述 7
2.3 RISC-V 交叉编译工具链的搭建和测试 8
2.3.1 编译工具链的基本工作流程 8
2.3.2 RISC-V 交叉编译工具链的安装与配置 9
2.3.3 Makefile 文件编写与编译测试 10
2.4 本章小结 11
3 基于国产平台的 tinyriscv 微控制器设计与移植 12
3.1 微控制器的整体架构与核心模块 12
3.2 处理器的流水线设计 13
3.2.1 IF 阶段（Instruction Fetch） 14
3.2.2 ID 阶段（Instruction Decode） 14
3.2.3 EX 阶段（Execute/Memory/Write Back） 14
3.3 基于紫光 FPGA 平台的 tinyriscv 移植 15
3.3.1 开发板资源介绍 15
3.3.2 移植过程及关键问题解决 16
3.3.3 存储模块适配调整 17
3.3.4 移植后资源消耗率 17
3.4 RTL 仿真运行 C 语言程序验证 18
3.5 微控制器功能仿真验证 19
3.6 PC 端与硬件交互工具 OpenOCD 20
3.6.1 OpenOCD 工作原理 20
3.6.2 OpenOCD 环境配置及应用 21

3.7 跑分测试 22
3.8 系统整体架构 22
3.9 本章小结 23
4 微控制器外设系统的设计与实现 24
4.1 RAM 和 ROM 24
4.2 TIMER 模块 24
4.2.1 TIMER 寄存器设计 25
4.2.2 TIMER 功能测试与验证 25
4.3 UART 模块 26
4.3.1 UART 寄存器设计 27
4.3.2 UART 功能测试与验证 27
4.4 SPI 模块 28
4.4.1 SPI 寄存器设计 29
4.4.2 SPI 功能测试与验证 29
4.5 GPIO 模块 30
4.5.1 寄存器设计 31
4.5.2 功能测试与验证 31
4.6 本章小结 32
5 基于 tinyriscv 的 RTOS 移植 33
5.1 RTOS 的简介 33
5.2 FreeRTOS 的移植 33
5.3 仿真验证与上板测试 34
5.4 本章小结 35
6 结论与展望 36
6.1 结论 36
6.2 展望 36
参考文献 37
致 谢 39
学位论文原创性声明 40
学位论文版权授权书 40

1 前言

1.1 研究背景及意义

随着信息技术的快速发展和集成电路制造工艺的不断进步，物联网（IoT, Internet of Things）作为新一代信息技术的重要组成部分，正在深刻改变社会生活和产业结构。大规模的智能设备与传感节点不断涌现，对高效、低功耗、可定制的微控制器提出了更高要求。随着晶体管于芯片里数量急剧增多以及集成密度不断提高，人类社会正渐渐步入“芯”时代，还为物联网应用打造了坚实的硬件根基。在这一发展潮流中，微控制器身为把中央处理器、存储器、输入输出模块等功能整合于一体的高效能控制单元，可承担复杂的计算与控制工作，在工业自动化、智能仪器、消费电子、物联网等领域被广泛运用，成为现代电子系统的关键构成部分。

自 20 世纪 40 年代晶体管诞生之后，半导体技术进入了快速发展时期。1971 年 Intel 公司推出首款微控制器 Intel 4004，这标志着微控制器技术正式踏入应用阶段，在此之后 MCS-51、ARM 架构等陆续出现，为嵌入式处理器的普及奠定了稳固基础，近些年，开源硬件理念的发展让 RISC-V 指令集架构受到了广泛关注。RISC-V 指令集架构依靠其开放、免费、精简、可扩展等优点，在学术界和产业界引发了广泛的关注[1]。RISC-V 降低了处理器设计门槛的同时，也为嵌入式系统和微控制器的创新应用提供了全新路径。

近年来，国家大力发展集成电路，CPU 产业已经得到显著发展，但是因为起步较晚，许多核心技术并没有真正掌握。并且以美国为首的发达国家为了遏制我国的发展，利用技术优势对我国半导体产业进行打压制裁，禁止为我国提供半导体核心产品和技术。为打造真正的国产可控且自主安全的集成电路，我们必须在指令集架构授权上摆脱国外企业的限制，减小芯片开发的难度，让中小型初创企业也能跨过芯片设计的门槛。在此背景下，RISC-V 指令集架构快速成为现代计算机体系结构研究以及实践操作的关键方向[2]，RISC-V 开源的特性给我国打破技术垄断、达成芯片自主创新给予了新契机。

从架构设计的发展趋向来看，传统的 CISC 架构像 x86 架构虽有着广泛应用基础，还拥有成熟的生态与稳定的性能，然而 x86 架构指令系统复杂且冗余致使设计与维护的成本较高。相较而言，RISC 架构依靠简化指令集提高了执行效率与系统扩展灵活性，渐渐成为嵌入式和低功耗设备的主要选择，ARM 架构作为 RISC 架构的代表长期主导移动和嵌入式市场，但其授权费用与闭源限制影响了企业的技术自主性，RISC-V 凭借指令集支持自由开放的指令集体系架构以及架构扩展来提供软件和硬件自由[3]，逐步成为嵌入式开发的新选项。

FPGA 作为一种高度可编程的数字集成电路，有良好的稳定性和并行数据处理能力[4]，它最大的特色是用户可在硬件层面自由定义并修改逻辑功能，与传统的 ASIC 相比，FPGA 在开发周期上大幅缩短，在硬件验证与功能调整方面成本更低且更具灵活性，FPGA 这种可重构的特性，使其广泛应用于原型设计、教学实验、科研探索等领域，特别适合作为处理器架构设计与验证的载体。

在这样的背景状况下，本文基于 FPGA 平台实现了一个 RISC-V 软核微控制器，FPGA 的灵活性与 RISC-V 的开放特性相结合，为研究自主指令集和嵌入式处理器提供了不错的平台支持，当前国内对自主可控芯片的需求日益增高，本设计有一定的现实意义，也有较强的技术可行性和工程应用价值。

1.2 国内外研究现状

基于 RISC-V 指令集架构设计的开源处理器具有开发周期短，支持商业模式的优点。对于知名的芯片企业和互联网公司以及个人和小型的公司开发均是非常友好的选择。

1.2.1 国外研究现状

2010 年，加州大学伯克利分校最早提出开源指令集架构——RISC-V 架构，目标是构建一套简洁、灵活、开放且具有可扩展性的指令集架构。RISC-V 凭借开源免费的特性迅速引起学术界与工业界的广泛关注。2015 年 RISC-V 基金会（现 RISC-V International）正式成立，进一步推动了 RISC-V 架构在软硬件开发领域的生态完善。随着生态系统的不断完善，Google、NVIDIA、Western Digital、Qualcomm、Samsung 等国际知名企业陆续加入基金会，并相继推出了自己基于 RISC-V 架构的各类芯片产品，预计到 2030 年，基于 RISC-V 的处理器将占据全球市场的近 1/4 的份额[5]。

RISC-V 架构已经在很多领域得到应用。比如，Western Digital 把大量硬盘控制器切换成了 RISC-V 架构。NVIDIA 在 AI 辅助模块中使用 RISC-V 架构做协处理器。得益于 RISC-V 开源的特性，很多初创公司也选择 RISC-V 架构来开发高性能处理器、边缘计算设备以及物联网芯片。此外 RISC-V 不仅在工业界受到关注，在学术界也相当受欢迎。很多高校用 RISC-V 作为计算机结构教学和研究的平台，同时 RISC-V 还推动了很多高校开展相关的创新项目和教学实验平台建设。

1.2.2 国内研究现状

在国家推动“国产可控”技术发展的背景下，RISC-V 架构在中国同样得到了迅速发展。近年来，高校和研究机构均积极开展基于 RISC-V 的处理器设计与软硬件协同研究，推动 RISC-V 架构在学术和产业两个方向的落地。在工业方面，阿里巴巴旗下的平头哥半导体推出了全球首款高性能 RISC-V 商用处理器——玄铁 910，最高工作频率超过 1GHz 已经在 AI、5G、边缘计算等多个场景获得实际应用[6-8]。在企业创新方面，华米科技自主设计了世界上第一款基于 RISC-V 用于穿戴的芯片“黄山一号”，该芯片拥有能效高、人工智能前移以及可以扩展等特点[9,10]。中科院孵化的芯来科技、兆易创新、华为等企业也相继基于 RISC-V 推出了自研内核和芯片产品，共同推动了国内自主芯片生态的完善。

RISC-V 架构为中国在芯片领域打破国外技术封锁提供了全新的突破口。RISC-V 开源和免授权费用的优势降低了芯片设计的门槛，为中小型芯片企业和科研单位提供了灵活高效的发展平台。国内对处理器架构自主权要求不断增强，RISC-V 已经在嵌入式系

统、智能终端、工业控制、汽车电子等领域得到广泛应用，逐步形成从指令集到工具链、从芯片到操作系统的完整生态体系。

1.2.3 发展趋势

当下 RISC-V 已然从实验室理论研究迈向实际应用阶段，正渐渐走入大众的视线范围，随着产业界对于 RISC-V 的关注度不断提升，其开发生态链也在持续壮大发展，未来随着交叉编译工具链、调试器以及操作系统不断完善，RISC-V 必定会在 AIoT 设备、边缘计算平台、低功耗嵌入式系统以及高性能计算机等诸多场景获得广泛应用。RISC-V 在灵活架构、自定义指令扩展以及降低开发成本等方面可契合更多差异化的应用需求，成为嵌入式芯片设计的关键方案之一。

1.3 主要研究内容与工作

本文围绕 RISC-V 指令集架构的软核微控制器展开研究，基于国产紫光同创 FPGA 平台完成了 tinyriscv 软核移的植工作，并搭建了完整的软硬件协同仿真验证平台。通过 RIB 总线将处理器、存储模块和外设模块连接，实现了软硬件的集成协同运行，完成了各模块的功能仿真、验证和上板测试。并且在 FPGA 平台上完成了整个微控制器系统的原型验证，并对其所占用的逻辑资源进行了统计与分析。最后基于该微控制器平台，成功移植轻量级实时操作系统 FreeRTOS，实现了多任务管理与调度功能。

本文结构安排如下：

第一章：简要介绍本设计的研究背景与意义，分析 RISC-V 指令集当前在国内外的发展现状及应用，为后续研究奠定基础。

第二章：系统阐述了 RISC-V 指令集架构的基本原理，回顾了指令集的发展历程，并探讨了微控制器实现的相关技术。完成了 RISC-V 交叉编译工具链的搭建。

第三章：研究了微控制器的整体架构与核心模块，详细分析了 tinyriscv 处理器内核的流水线结构及其关键逻辑模块，同时针对移植至国产“紫光 FPGA”平台过程中涉及的架构差异进行了相应修改与适配。完成了交互工具 OpenOCD 的搭建并通过简单的 C 语言程序对其进行了验证。

第四章：完成了微控制器外设系统的设计与实现，开发了 GPIO、SPI、UART 等基础外设模块，并通过仿真波形观察与板级测试验证了各模块的功能完整性与接口一致性。

第五章：简要介绍实时操作系统的基本概念与发展，围绕 FreeRTOS 的特点与结构展开研究，完成了其在本平台上的移植工作，并通过一个简单的例子进行了测试与验证。

第六章：对整个工作的总结与归纳，对其中存在的不足之处进行了分析，对可以改进的地方进行了展望。

2 相关技术背景

2.1 指令集架构介绍

指令集架构（Instruction Set Architecture，ISA）是连接软件和硬件之间的桥梁，是通过文档的形式发布的一套标准规范，包含一系列用机器语言编写的操作码以及由特定处理器执行的基本命令，是计算机体系架构的核心内容，也被称为指令系统。ISA 决定了高级语言代码通过特定的编译器生成的机器码格式，同时也规定了硬件需要如何响应这些指令。因此指令集不仅影响计算机的运行效率，还直接决定了系统的兼容性和可扩展性。

2.1.1 发展概述

20 世纪 50 年代指令集架构的概念产生，但由于当时的计算机通常由体积较大的电子管或晶体管组成，只包括十几条基本的指令，指令系统运行效果较差且速度较慢。随着半导体行业的发展，计算机集成度越来越高，功能也越来越完善和复杂，指令系统也随之变的多样化。从体系结构方面来看，指令集大致可划分成两大类，即复杂指令集与精简指令集，CISC 架构的典型代表像 Intel x86 系列，它有指令种类繁杂、格式复杂的特性，凭借一条指令去完成更多操作，指令长度一般不相等，与之相比，RISC 架构主张把指令控制在较少数量，并且让指令格式保持简洁统一，借助多条精简指令组合来达成复杂功能，对处理器的设计给予简化，提升执行效率以及并行度。

在发展进程中，CISC 和 RISC 在设计理念上相互学习借鉴，二者的界限变得越来越模糊，像传统的 CISC 架构代表 x86 处理器在近些年引入了类似 RISC 的微操作机制：尽管程序员看到的是复杂的 CISC 指令，但实际上在处理器内部这些指令会被拆解为更简单的微指令然后由流水线执行，这样的方式大幅提高了执行效率和并行度。另一方面，RISC 架构中的 ARM 在最新的处理器设计里也引入了一些复杂指令以及执行优化机制，比如 ARM 的 Thumb 指令集压缩机制以及后续的 ARMv8-A 指令集中引入的高级矢量扩展，都体现出其设计朝着性能与灵活性平衡的方向发展。目前主流的指令集架构如表 2.1 所示。

表 2.1 主流指令集架构

架构名称	运营机构	发布时间	架构特点	代表厂商
X86	美国英特尔公司	1978 年	性能高、速度快、兼容好	英特尔、AMD
MIPS	美国 MIPS 科技公司	1981 年	简洁、高拓展性	龙芯
ARM	英国 ARM 公司	1983 年	低成本、低功耗	苹果、华为
RISC-V	RISC-V 基金会	2011 年	模块化、可裁剪	三星、英伟达

近年来，随着嵌入式系统与物联网的快速发展，轻量级、高性能、低功耗的处理器需求愈加迫切，RISC 架构在工业和学术界受到广泛青睐。RISC-V 作为一种新兴的开放

式精简指令集架构，因其免费、开放、模块化、可扩展等特点，迅速形成了丰富的生态系统，成为当前指令集架构研究与应用的热点方向。RISC-V 基金会的成员当下已超过 335 家公司或组织，我国的阿里巴巴和华为也为基金会成员，图 2-1 为部分 RISC-V 基金会成员[11]。

图 2-1 部分 RISC-V 基金会成员

2.1.2 RISC-V 指令集架构

RISC-V 是一套源自学术界且面向工业实际应用的开源指令集架构，其架构秉持模块化设计与简洁性的核心理念，有依据应用需求灵活进行裁剪与扩展的能力，拥有良好的可移植性以及可维护性[12,13]。

RISC-V 指令集架构的基础部分为整数指令集，简称为“I”指令集，按照处理器的数据宽度差异，被划分成 RV32I、RV64I 以及 RV128I 版本。RV32I 在嵌入式系统中有着广泛应用，适用于资源受限但对稳定性有着较高要求的场景，RISC-V 还推出了 RV32E 指令集版本来契合对芯片面积与功耗较为敏感的嵌入式应用，RV32E 把通用寄存器数量从标准的 32 个减少至 16 个，在降低硬件资源消耗的还可以保证指令集的基本功能，适用于小型设备或者对成本敏感的系统设计。除了基础的整数指令外，RISC-V 还定义了多种标准扩展指令集，这些扩展可按需进行选择组合，帮助设计者依照实际需求灵活调整处理器功能，常用的扩展有：

（1）M 扩展（Multiplication & Division）：其提供了标准的整数乘除法指令，以此来契合基本运算方面的需求；

（2）A 扩展（Atomic）：它增加了原子读-修改-写操作，提升处理器数据同步能力；

（3）C 扩展（Compressed）：能提供长度为 16 位的压缩指令，以此提升代码密度，减少指令存储空间的占用，还可以优化指令缓存利用率；

（4）F/D/Q 扩展（Floating-Point）：分别对单精度 32 位、双精度 64 位以及四倍精度 128 位的浮点数运算提供支持，适用于科学计算与信号处理；

（5）其他扩展包含向量扩展 V、加密扩展 B、预取扩展 P 等，这些扩展指令提高了 RISC-V 架构的适用范围以及灵活性。

在指令编码设计这一方面，RISC-V 采用固定长度的指令，如此一来，解码器的复杂度得以简化，处理器的流水线执行效率也有所提升，不同类型的指令有着 R、I、S、B、U、J 六种编码格式，它们分别对应着寄存器-寄存器操作、短立即数与访存 load 操作、访存 store 操作、条件跳转操作、长立即数操作、无条件跳转操作。指令格式遵循统一的规范，而且操作字段一直处在固定的位置，就好比，寄存器的标识符在所有指令里的位置是保持一致的，这利于硬件设计提前去访问寄存器，提升解码和执行的效率，指令中的立即数字段都采用符号扩展方式，符号位固定于最高位，这样就简化了数据解释逻辑，使得指令处理的稳定性得到提升[14]。各种类型的指令构成如图 2-2 所示。指令中各字段的含义如表 2.2 所示。

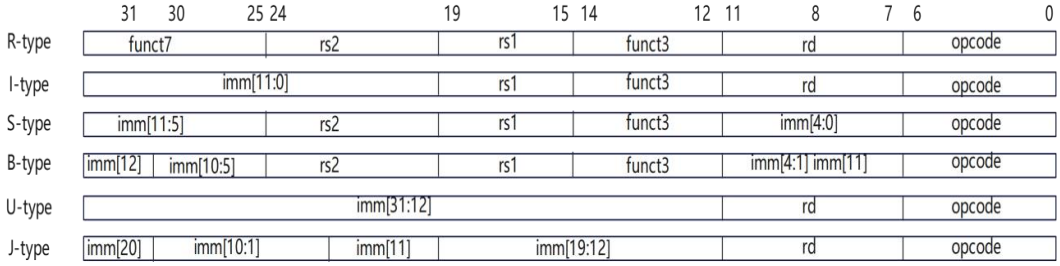


图 2-2 RISC-V 指令格式示意图

表 2.2 RISC-V 指令中各字段含义

字段	含义
opcode	指令的基本操作，这个缩写是它惯用名称。
rd	目的操作寄存器，用来存放操作结果。
funct3	一个另外的操作码字段
rs1	第一个源操作数寄存器
rs2	第二个源操作数寄存器
funct7	一个另外的操作码字段
imm	立即数

RV32I 指令集包含算术运算、逻辑运算、数据传输、跳转控制以及系统调用等基础操作，可契合通用计算架构下的功能需求，该架构还借助规范的立即数编码方式，简化了符号扩展与硬件译码逻辑，提高了指令执行的效率及可预测性。

本项目采用的处理器核心基于 RISC-V 架构的 RV32I 指令集，结合具体外设接口需求，完成了软硬件协同设计，构建了完整的微控制器功能框架。深入理解指令集架构后，保证软核能稳定运行，契合系统基本功能要求，也为后续操作系统的移植与平台扩展奠定了良好基础，为更直观地了解其技术与系统地位，RISC-V 在计算机系统中的结构如图 2-3 所示。

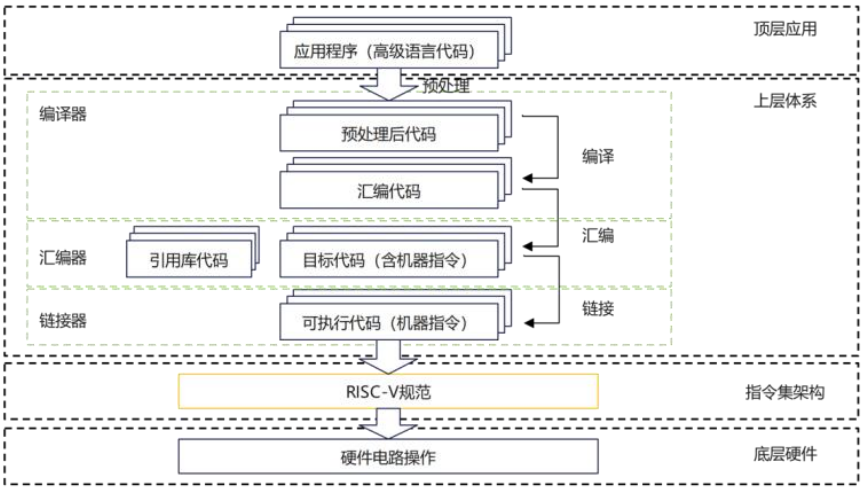


图 2-3 RISC-V 在系统中的架构图

2.2 FPGA 概述

FPGA 即现场可编程门阵列，是一种集成电路，可依靠软件配置来定义硬件功能，用户完成设计后，可多次对芯片编程与修改，迅速实现逻辑功能的调整和优化，相较于传统专用集成电路 ASIC，FPGA 有设计灵活、开发周期短以及可重复编程的优势，能契合不同应用场景需求。

FPGA 拥有良好的并行处理能力，芯片内部含有大量可编程逻辑单元与片上存储资源，可同时执行多路数据的计算任务，此特性让 FPGA 在通信系统、嵌入式平台、人工智能加速、图像处理以及高性能计算等领域得以广泛应用。在数据吞吐量与处理速度方面，FPGA 比传统通用处理器表现更优，特别适合对实时性和并行性要求较高的应用场景。从硬件结构来看，FPGA 主要由可编程输入输出单元 I/O、可编程逻辑块 LC、时钟管理模块 CMT、片上嵌入式 RAM 即 BRAM、可编程互连资源以及部分内嵌的底层功能模块构成，另外部分型号的 FPGA 芯片还集成了专用硬件 IP 模块，像高速接口控制器、数字信号处理单元 DSP 等，以契合不同领域应用需求。

FPGA 的开发流程一般包含四个主要阶段，设计者依据功能需求，完成算法分析与模块设计，使用硬件描述语言 HDL 如 Verilog 或 VHDL，完成逻辑功能的代码实现，随后利用 EDA 工具对设计文件进行仿真，保证功能正确，经过综合、布局布线后，生成比特流文件 Bitstream，依靠下载器将程序烧录到 FPGA 芯片中，实现硬件功能的部署[15]。

国际主流的 FPGA 厂商主要有 Xilinx 和 Altera，这两家厂商分别推出了成熟的设计工具与多款 FPGA 产品系列，比如 Xilinx 的 Vivado 开发平台，以及 Altera 的 Quartus Prime 工具，广泛应用于各类工业和科研场景，除国外厂商外，国产 FPGA 产业近年来不断发展，逐渐打破长期对国外芯片的依赖。紫光同创、安路科技、复旦微电子等国内企业已相继推出多款自主研发的 FPGA 产品，逐步建立起完整的芯片设计、开发工具链和应用生态体系，国产 FPGA 的出现，降低了硬件开发的采购成本，还提升了本土芯片在关键领域的自主可控能力，帮助我国芯片产业链完善。

FPGA 作为一种灵活且高效的可编程逻辑器件，可为电子系统设计奠定稳定可靠的硬件基础，随着国产 FPGA 厂商持续发展壮大，未来 FPGA 在嵌入式系统、工业控制、人工智能加速以及芯片原型验证等领域会发挥更为关键的作用，有着广阔的市场前景与应用价值。

此次设计采用紫光同创 PGL22G 系列 FPGA 当作软核微控制器的实现平台。该系列芯片有丰富逻辑资源，时序特性良好，可契合软核微处理器架构运行需求，借助 FPGA 可编程特性，能快速完成硬件逻辑功能验证与系统迭代，缩短设计周期，提升开发效率，FPGA 平台还为软硬件联合调试创造了良好实验条件，保证 RISC-V 软核架构功能稳定以及移植适配正确。芯片情况如图 2-4 所示。

图 2-4 紫光同创 PGL22G 芯片实物图

2.3 RISC-V 交叉编译工具链的搭建和测试

2.3.1 编译工具链的基本工作流程

计算机上进行编程时，我们通常使用如 C 语言、Python 等高级语言。但是计算机本质上只能识别运行机器语言，即二进制代码。高级语言编写的程序需要经过一系列处理过程，最终转换为可执行的机器指令。整个转换过程主要包括四个阶段：预处理（Preprocessing）、编译（Compiling）、汇编（Assembling）和链接（Linking）。以下为该四个阶段功能及流程的简要介绍：

（1）预处理：编写好的 C 语言源代码首先会经过预处理阶段，此阶段的主要任务包括：将代码中包含的头文件内容插入到源文件中；对宏定义进行替换；删除注释内容和无用代码等。预处理后生成的是一个扩展的源文件，便于后续的编译过程。

（2）编译：于编译阶段的时候，编译器会针对预处理之后的源文件开展词法分析、语法分析以及优化工作，把高级语言代码转变为对应的汇编代码，此步骤乃是把抽象的语言逻辑翻译成为更加贴近底层硬件的指令集合。

（3）汇编：汇编过程承担着把汇编语言翻译为机器码的任务，生成目标文件。在这一阶段，会将汇编语言里有可读性的符号指令，转化为计算机可直接识别的二进制指令。

（4）链接：链接器将各个目标文件与程序中所依赖的库文件进行链接，统一整合成一个完整的可执行文件，如.elf、.bin 等。该文件不仅包含了程序的机器指令，还包含了运行所需的入口地址、加载地址、符号表、数据段等必要信息，使得程序能够被正确加载并在系统中运行。

2.3.2 RISC-V 交叉编译工具链的安装与配置

在开展嵌入式系统开发或者异构平台开发工作时，一般都要用到交叉编译工具链，来在开发主机上为目标平台编译产出可执行文件。鉴于目标平台的硬件架构大多时候和主机不一样，比如主机采用 x86 架构，而目标设备运用 RISC-V 架构，本地编译器没办法直接生成适合目标平台的可执行文件。交叉编译工具链可处理架构差异问题，实现针对目标平台的程序生成，是嵌入式开发里不可或缺的关键工具[16]。

在 RISC-V 架构的开发进程中，最为常用的交叉编译工具链是由 riscv-gnu-toolchain 项目供应的，针对不同的运行环境，工具链前缀的命名也存在差异。举例来讲：riscv-none-embed-前缀的工具链，主要是面向裸机环境进行开发，在裸机开发模式下，程序直接在硬件上运行，不依靠操作系统，一般用于 BootLoader、驱动程序、简单应用以及轻量级实时操作系统等场景，这个版本的工具链默认集成了 newlib 标准 C 库，部分版本支持 newlib-nano，相较于传统标准库，newlib-nano 可有效减小程序体积，降低资源占用，对于存储空间有限的嵌入式设备更为适宜。

本文中采用的处理器架构属于典型的嵌入式裸机系统，参考开源项目 tinyriscv，因此选用以 riscv-none-embed-为前缀的 GCC 交叉编译工具链作为开发环境。在交叉编译过程中，可以通过 GCC 的-march 编译参数指定目标平台所支持的指令集。例如：rv32i[m][a][f][d][c]：表示 32 位 RISC-V 指令集，支持不同的扩展。tinyriscv 软核架构支持 RV32IM 指令集，即基础整数指令加乘除法扩展。所以-march 参数设定为 rv32im。同时，由于硬件不支持浮点运算，ABI 参数配置选择-mabi=ilp32，保证编译生成的程序在 32 位架构平台下正确运行。在 Windows 开发环境下安装完成 RISC-V 交叉编译工具链后，需要将工具链的 bin 目录路径添加到系统环境变量 PATH 中，这样可以在命令行终端中直接调用编译工具进行代码编译与调试。高级语言程序具体转换流程如图 2-5 所示。

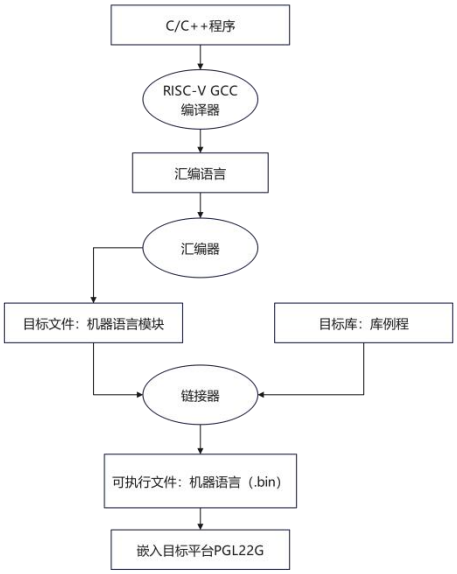


图 2-5 C/C++到机器语言过程示意图

RISC-V 工具链中常用的一些工具功能介绍如表 2.3 所示：

表 2.3 riscv-gnu-toolchain 工具链常用工具及功能

工具链种类	介绍	功能
riscv-none-embed-gcc	RISC-V 交叉编译器	C 语言编译器，编译为目标文件
riscv-none-embed-g++	RISC-V 交叉编译器	C++语言编译器，编译为目标文件
riscv-none-embed-as	RISC-V 汇编器	将汇编源文件.s 编译为目标文件.o
riscv-none-embed-ld	RISC-V 链接器	链接目标文件与库文件生成.elf
riscv-none-embed-objcopy	目标文件格式转换工具	将.elf 等文件转换为.bin 格式
riscv-none-embed-objdump	反汇编工具	将二进制文件转换为汇编代码
riscv-none-embed-gdb	RISC-V 调试器	对程序进行调试

2.3.3 Makefile 文件编写与编译测试

完成工具链搭建后，编写了简单的测试文件 test.c，并配套链接脚本 link.lds。link.lds 文件用于指导链接器将各个目标文件按照指定的内存布局进行组织，确保程序中的各个代码段、数据段能够正确映射到目标平台的存储空间。链接脚本通常定义了程序的入口地址、代码段的起始位置、堆栈空间分配以及各类数据段的加载地址，确保生成的可执行文件能够在嵌入式硬件上按照预期进行运行。

在本设计的链接脚本中，主要完成了以下工作：指定了程序入口函数的地址，划分了.text（代码段）、.data（已初始化数据段）和.bss（未初始化数据段）等常用段的存储位置，同时为堆栈预留了固定空间，避免程序运行过程中出现地址冲突。通过合理的地址布局，确保编译生成的可执行文件能够被正确加载并在 tinyriscv 软核平台上运行。

为了便捷编译的过程，避免反复的手动输入复杂的命令，因此编写了 Makefile 文件，统一管理代码的编译、链接以及可执行文件的生成过程。

有了 Makefile 后，只需要在对应目录的命令行中通过 make 即可快速完成编译，也可以通过 make clean 命令进行清理中间文件的生成。图 2-6 为执行 make 命令后编译的过程。Makefile 主要通过定义变量的方式，完成编译参数、目标文件名称、用到的源文件路径等内容的配置。文件中定义了目标平台的架构类型、ABI 规范以及内存模型，确保编译器可以根据目标硬件平台的特性正确生成可执行文件。并且 Makefile 还设置了代码优化等级与仿真环境的宏定义，方便程序在不同环境的灵活切换。在该文件的末尾，通过调用 common.mk 将每个工程通用的编译内容分离出来，提高了不同工程目录之间灵活调用。

使用 riscv-gnu-toolchain 工具链配合 Makefile 完成代码的编译与链接后，成功生成了适用于目标平台的可执行文件。如图 2-7 所示，通过测试验证，工具链安装正确，编译流程稳定。

```
PS C:\Users\BOSS\Desktop\Graduation\liangkangnan\tinyriscv\tests0\example\timer_int> make clean
rm -f timer_int ../start.o ../trap_entry.o main.o ../init.o ../trap_handler.o ../lib/utils.o ../lib/xprintf.o ../lib/uar
t.o ../lib/flash_n25q.o timer_int.dump timer_int.bin
PS C:\Users\BOSS\Desktop\Graduation\liangkangnan\tinyriscv\tests0\example\timer_int> make
C:/Users/BOSS/Desktop/Graduation/liangkangnan/tinyriscv/tools/gnu-mcu-eclipse-riscv-none-gcc-8.2.0-2.2-20190521-0004-win
64/bin/riscv-none-embed-gcc -DSIMULATION -Os -march=rv32im -mabi=ilp32 -mcmodel=medlow -ffunction-sections -fdata-sectio
ns -fno-builtin-printf -fno-builtin-malloc -I.. -c -o ../start.o ../start.S
C:/Users/BOSS/Desktop/Graduation/liangkangnan/tinyriscv/tools/gnu-mcu-eclipse-riscv-none-gcc-8.2.0-2.2-20190521-0004-win
64/bin/riscv-none-embed-gcc -DSIMULATION -Os -march=rv32im -mabi=ilp32 -mcmodel=medlow -ffunction-sections -fdata-sectio
ns -fno-builtin-printf -fno-builtin-malloc -I.. -c -o ../trap_entry.o ../trap_entry.S
C:/Users/BOSS/Desktop/Graduation/liangkangnan/tinyriscv/tools/gnu-mcu-eclipse-riscv-none-gcc-8.2.0-2.2-20190521-0004-win
64/bin/riscv-none-embed-gcc -DSIMULATION -Os -march=rv32im -mabi=ilp32 -mcmodel=medlow -ffunction-sections -fdata-sectio
ns -fno-builtin-printf -fno-builtin-malloc -I.. -c -o main.o main.c
C:/Users/BOSS/Desktop/Graduation/liangkangnan/tinyriscv/tools/gnu-mcu-eclipse-riscv-none-gcc-8.2.0-2.2-20190521-0004-win
64/bin/riscv-none-embed-gcc -DSIMULATION -Os -march=rv32im -mabi=ilp32 -mcmodel=medlow -ffunction-sections -fdata-sectio
ns -fno-builtin-printf -fno-builtin-malloc -I.. -c -o ../init.o ../init.c
C:/Users/BOSS/Desktop/Graduation/liangkangnan/tinyriscv/tools/gnu-mcu-eclipse-riscv-none-gcc-8.2.0-2.2-20190521-0004-win
64/bin/riscv-none-embed-gcc -DSIMULATION -Os -march=rv32im -mabi=ilp32 -mcmodel=medlow -ffunction-sections -fdata-sectio
ns -fno-builtin-printf -fno-builtin-malloc -I.. -c -o ../trap_handler.o ../trap_handler.c
C:/Users/BOSS/Desktop/Graduation/liangkangnan/tinyriscv/tools/gnu-mcu-eclipse-riscv-none-gcc-8.2.0-2.2-20190521-0004-win
64/bin/riscv-none-embed-gcc -DSIMULATION -Os -march=rv32im -mabi=ilp32 -mcmodel=medlow -ffunction-sections -fdata-sectio
ns -fno-builtin-printf -fno-builtin-malloc -I.. -c -o ../lib/utils.o ../lib/utils.c
C:/Users/BOSS/Desktop/Graduation/liangkangnan/tinyriscv/tools/gnu-mcu-eclipse-riscv-none-gcc-8.2.0-2.2-20190521-0004-win
64/bin/riscv-none-embed-gcc -DSIMULATION -Os -march=rv32im -mabi=ilp32 -mcmodel=medlow -ffunction-sections -fdata-sectio
ns -fno-builtin-printf -fno-builtin-malloc -I.. -c -o ../lib/xprintf.o ../lib/xprintf.c
C:/Users/BOSS/Desktop/Graduation/liangkangnan/tinyriscv/tools/gnu-mcu-eclipse-riscv-none-gcc-8.2.0-2.2-20190521-0004-win
64/bin/riscv-none-embed-gcc -DSIMULATION -Os -march=rv32im -mabi=ilp32 -mcmodel=medlow -ffunction-sections -fdata-sectio
ns -fno-builtin-printf -fno-builtin-malloc -I.. -c -o ../lib/uart.o ../lib/uart.c
C:/Users/BOSS/Desktop/Graduation/liangkangnan/tinyriscv/tools/gnu-mcu-eclipse-riscv-none-gcc-8.2.0-2.2-20190521-0004-win
64/bin/riscv-none-embed-gcc -DSIMULATION -Os -march=rv32im -mabi=ilp32 -mcmodel=medlow -ffunction-sections -fdata-sectio
ns -fno-builtin-printf -fno-builtin-malloc -I.. -c -o ../lib/flash_n25q.o ../lib/flash_n25q.c
C:/Users/BOSS/Desktop/Graduation/liangkangnan/tinyriscv/tools/gnu-mcu-eclipse-riscv-none-gcc-8.2.0-2.2-20190521-0004-win
64/bin/riscv-none-embed-gcc -DSIMULATION -Os -march=rv32im -mabi=ilp32 -mcmodel=medlow -ffunction-sections -fdata-sectio
ns -fno-builtin-printf -fno-builtin-malloc -I.. ../start.o ../trap_entry.o main.o ../init.o ../trap_handler.o ../lib/uti
ls.o ../lib/xprintf.o ../lib/uart.o ../lib/flash_n25q.o -o timer_int -T ../link.lds -nostartfiles -Wl,--gc-sections -Wl,
--check-sections
C:/Users/BOSS/Desktop/Graduation/liangkangnan/tinyriscv/tools/gnu-mcu-eclipse-riscv-none-gcc-8.2.0-2.2-20190521-0004-win
64/bin/riscv-none-embed-objcopy -O binary timer_int timer_int.bin
C:/Users/BOSS/Desktop/Graduation/liangkangnan/tinyriscv/tools/gnu-mcu-eclipse-riscv-none-gcc-8.2.0-2.2-20190521-0004-win
64/bin/riscv-none-embed-objdump --disassemble-all timer_int > timer_int.dump
```

图 2-6 执行 make 命令

图 2-7 工具链测试生成的文件

2.4 本章小结

本章主要介绍了本设计所用到的相关技术背景，包括指令集架构、FPGA 以及 RISC-V 交叉编译工具链的实现过程，为后续的软核移植工作奠定了理论基础。首先对指令集架构的发展进行概述，同时从 RISC-V 指令集的版本、基础指令集、扩展指令集以及指令格式等方面说明了该指令集的特点。其次通过对 FPGA 进行了介绍，包括特点、开发流程、常见 FPGA 产品的描述同时引出本设计的实现平台。最后对本设计中交叉编译工具链的搭建和测试做了详细介绍，对编译工具链的基本工作流程、交叉编译工具链的安装与配置和 Makefile 文件编写与编译测试进行全面分析。

3 基于国产平台的 tinyriscv 微控制器设计与移植

3.1 微控制器的整体架构与核心模块

本设计是基于 RISC-V 架构的 tinyriscv 软核设计，并结合国产紫光 FPGA 平台的实际资源进行裁剪和优化。软核设计以模块化的思路可划分为：获取指令（IF）、指令译码（ID）、执行运算（EX）、访存操作（MEM）和结果写回（WB）。各功能模块之间通过总线互联的方式进行信号传输，实现模块之间的协同工作。在系统架构上，采用哈佛结构划分指令存储空间与数据存储空间，指令由 ip_dual_port_ram 作为 ROM 载体存储，数据则通过同样的 RAM 结构进行读写。核心模块之间通过总线接口互联，实现了软硬件之间的信号兼[17,18]容。具体架构如图 3-1 所示。

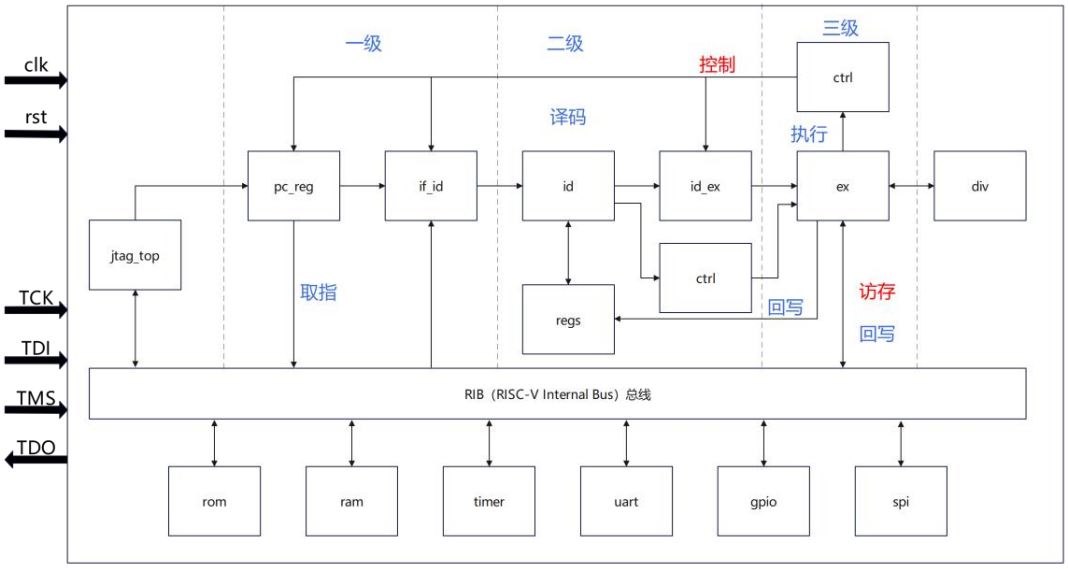


图 3-1 tinyriscv 系统结构框图

图中主要模块的介绍：

jtag_top：调试的顶层模块，主要有三大类型的信号，第一种是读写内存的信号，第二种是读写寄存器的信号，第三种是控制信号，例如复位 MCU、暂停 MCU 等操作。

pc_reg：PC 寄存器模块，用于产生 PC 寄存器的值，该值会被用作指令存储器的地址信号，并根据跳转和暂停等信号控制调整 PC 的值。

if_id：取指到译码之间的流水线寄存器模块，用于将指令存储器输出的指令打一拍后稳定传递至译码模块，确保数据时序正确。

id：译码模块，采用纯组合逻辑，根据 if_id 模块送进来的指令内容完成指令译码。当译码出具体的指令(比如 add 指令)后，产生是否写寄存器信号，读寄存器信号等。由

于寄存器采用的是异步读方式，因此只要送出读寄存器信号后，会马上得到对应的寄存器数据，这个数据会和写寄存器信号一起送到 id_ex 模块。

id_ex：译码到执行之间的流水线寄存器模块，用于将 ID 阶段输出的控制信号和操作数稳定地传递到 EX 阶段，确保指令正确执行。

ex：执行模块，纯组合逻辑电路，根据具体的指令进行相应的操作，如是 add 等运算指令则执行加法等运算操作；如是 lw 等访存指令，则会进行读内存操作，读内存将采用异步读的方式。最后将是否需要写寄存器、写寄存器地址以及写寄存器数据信号传送给 regs 模块，然后将其传递至 rib 总线，由总线来分配访问的模块。

div：除法模块，采用试商法算法，通常需要至少 32 个时钟周期完成一次除法操作，用于处理带除法运算的 RISC-V 指令。

ctrl：控制模块，负责产生控制信号，包括暂停流水线、跳转请求、异常处理等。

clint：核心本地中断模块，对输入的中断请求信号进行集中管理，产生最终的中断信号并向处理器核发出中断响应。

在开展 SoC 系统设计工作时，要是缺失总线架构，处理器核跟外设之间的连接会变得极为繁杂，每增添一个外设，处理器都得单独构建一条地址总线以及一条数据总线，假定系统含有 N 个外设，处理器就得维护 N 条地址总线与 N 条数据总线，硬件连线数量会随着外设数量呈线性增长态势。此种方式加大了系统的硬件复杂程度，而且在后期对外设进行修改时，处理器核心的代码也得同步给予调整，给系统维护以及扩展给予了不小的负担。为简化处理器核与外设之间的连接关系，软核设计里引入了总线结构，借助引入总线，处理器核只需经由一组统一的地址总线和数据总线与外设展开交互，由总线模块负责具体的地址译码与访问分发工作，较大降低了硬件连线的复杂程度，还提升了系统架构的可扩展性。

当前业界存在多种成熟且标准的总线协议，像 AMBA、Wishbone、AXI 等，这些协议可契合大部分系统设计的需求，并且有丰富的开发资源，设计者可直接选用这些总线规范来加快开发流程，在本设计中，鉴于软硬件协同调试的需求，采用了一套 RISC-V 自主的总线架构，称作 RIB。RIB 总线支持多主多从设备连接，不过在同一时刻只允许一对主从设备进行通信，以此保证系统访问的有序性，而且 RIB 总线内部采用固定优先级仲裁机制，用以解决多主设备同时发起访问请求时出现的冲突问题，借助引入 RIB 总线，达成了软核内部核心模块与外设模块之间的标准化互联，降低了整体系统的设计复杂程度，还为后续的模块扩展提供了便利条件。

3.2 处理器的流水线设计

在嵌入式处理器设计之时，由于硬件资源有限，故而要在控制复杂度、性能以及资源消耗这几方面达成平衡。Tinyriscv 软核运用了三级流水线结构，以此来适配中小型 FPGA 平台的资源状况，同时兼顾系统运行效率以及设计的可维护性，tinyriscv 的三级流水线主要划分成以下阶段：

3.2.1 IF 阶段（Instruction Fetch）

处理器通过程序计数器（PC）从指令存储器中读取下一条需要执行的指令，并将其传递给流水线的下一阶段，当到达时钟上升沿时，PC 会根据系统状态更新：①如果系统正常运行，PC 自己会按照顺序递增，每次加 4 以逐条读取指令；②如果接收到跳转信号，PC 值会更新为跳转地址，进行程序流程的切换；③如果处于暂停状态，保持 PC 不变。

通过这种流程，IF 阶段可以从 ROM 中获取指令，并通过流水线寄存器 if_id 将指令传递到 ID 阶段，一级一级将指令传递。

3.2.2 ID 阶段（Instruction Decode）

完成指令的译码操作，解析指令格式，识别指令类型，并从寄存器堆中读取源操作数。ID 阶段还会根据不同指令的特征，生成对应的控制信号，驱动后续执行阶段的逻辑运算或访存操作。以 ADD 指令为例说明译码流程。如图 3-2 所示是 ADD 指令的编码格式。

	1	2	3	4	5	6
ADD	000000	rs2	rs1	000	rd	0110011

图 3-2 ADD 指令的编码格式

ADD 指令被编码分为 6 部分。通过 1、4、6 这三部分可以确定当前指令是 ADD 指令，然后就可以知道 ADD 指令需要读两个通用寄存器（rs1 和 rs2）和写一个通用寄存器（rd）。

3.2.3 EX 阶段（Execute/Memory/Write Back）

根据指令类型执行算术或逻辑运算；对于访存类指令，执行数据的读写操作；对于普通计算指令，将运算结果写回目标寄存器。指令的大部分功能在这一阶段完成，并在执行结束后将结果写回寄存器，或发出访存请求，通过总线读写数据。

（1）算术与逻辑运算

对于普通计算型指令（如 ADD、SUB、AND 等），执行模块会根据译码阶段传递的控制信号，调用相应的运算单元进行计算。例如，对于 ADD 指令，执行阶段的硬件逻辑会根据以下公式完成运算：add_sub_res=add_op1+(op_sub?(-add_op2):add_op2).信号 op_sub 控制加法或减法操作（op_sub=0，执行加法；相反则执行减法）。这种设计方式将加法和减法的运算逻辑整合为一个通用加法器，节省了硬件资源并简化了设计复杂度。

（2）跳转判断与地址计算

对于分支或跳转类指令（如 BEQ、BNE、JAL 等），执行阶段会根据寄存器中的值完成条件判断，并计算跳转目标地址，若条件满足，将跳转地址通过控制信号传递回 PC 模块，完成程序流的切换。

（3）访存操作

对于 LOAD 和 STORE 类指令，执行阶段会完成访存地址的计算，并根据指令类型发出读写请求，数据通过 RIB 总线与 RAM 模块交互，完成存储器的数据访问。

三级流水线工作流程如图 3-3 所示。

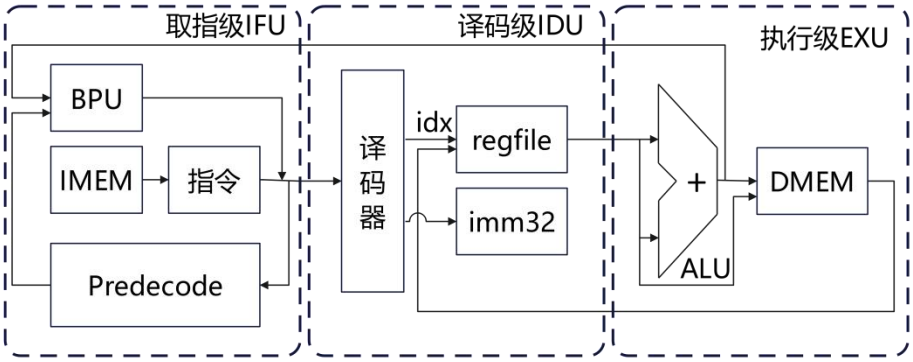


图 3-3 tinyriscv 三级流水线工作流程图

把执行、访存以及写回阶段给予合并，三级流水线设计切实减少了硬件逻辑的复杂程度，降低了指令级之间传输的时间延迟，提升了系统运行的可靠性能。和五级流水线相比较而言，三级结构处理冒险情况更为简便，减少了插入气泡的可能性，同时规避了复杂的数据前递逻辑，降低了硬件资源的消耗，特别适用于国产 FPGA 平台的逻辑面积及时序约束条件[19]。

Tinyriscv 软核采用的三级流水线设计，在维持较低资源占用的情况下，可契合基本的指令执行效率，适宜在嵌入式系统和教学研究等场景中使用，为后续的软件移植以及硬件扩展提供了不错的架构基础。

3.3 基于紫光 FPGA 平台的 tinyriscv 移植

3.3.1 开发板资源介绍

Logos 系列可编程逻辑器件是深圳市紫光同创电子有限公司研发的全新低功耗、低成本 FPGA 产品，它采用了完全自主产权的体系结构和主流的 40nm 工艺。本项目选用的 FPGA 平台是紫光同创的 PGL22G FPGA 开发板，其具有丰富的硬件资源，能够满足微控制器移植及后续功能拓展的需求。具备 26304 个触发器（FF）、17536 个查找表 LUT5（等效 LUT4 数量为 21043）、48 个 DRM（18Kbits）、30 个 APM 单元（乘法器）以及 1 个模数转换/ADC，速度等级为-6，温度等级为工业级，能够为微控制器的运行提供强大的硬件支持。开发板上配备了一片 16bit 数据位宽的 DDR3，总容量高达 512Mbit，整个系统的存储带宽高达 800Mb/s，充分满足了高速数据的缓存处理需求。另外还有一片 128Mb QSPI Flash，作为微控制器的默认启动方式，用于存储程序代码和相关数据。并且提供了丰富的外设接口，包括一路光纤通信模块接口、一路扩展的差分 40PIN IO 接口、一路千兆以太网接口、一路 HDMI 输入接口、一路 HDMI 输出接口、两路 PMOD（Camera）2*6PIN 接口、一路 SD 卡接口、一路 Uart 接口等，为微控制器与其他设备的通信和功能扩展提供了便利。基于以上特点，Logos 系列 FPGA 能够广泛适用于视频、工业控制、汽车电子和消费电子等多个应用领域[20]。开发板实物如图 3-4 所示。

图 3-4 紫光同创 PGL22G 开发板实物图

3.3.2 移植过程及关键问题解决

在本设计中，为了将基于 RISC-V 指令集架构的 tinyriscv 微控制器成功移植至紫光同创 PGL22G FPGA 平台，需要解决关键问题是存储器的实现方式差异。开源的 tinyriscv 工程中，存储模块采用 reg 数组在 Verilog 中直接建模。这种方式在仿真阶段能够正常运行，但在 FPGA 实际部署时，尤其是在 DRAM 资源有限的情况下，无法满足对存储空间的需求。因此针对紫光同创 PGL22G 开发板，必须将存储器部分替换为基于 BRAM（Block RAM）IP 核实现的方式[21]。

在移植过程中，参考了 tinyriscv 项目中 BRAM 分支，并根据紫光平台提供的 IP 核特性，将原设计中调用的通用 BRAM IP 核替换为紫光同创官方提供的 ip_dual_port_ram IP 核，实现存储资源的合理配置。IP 核的设置如图 3-5 所示。

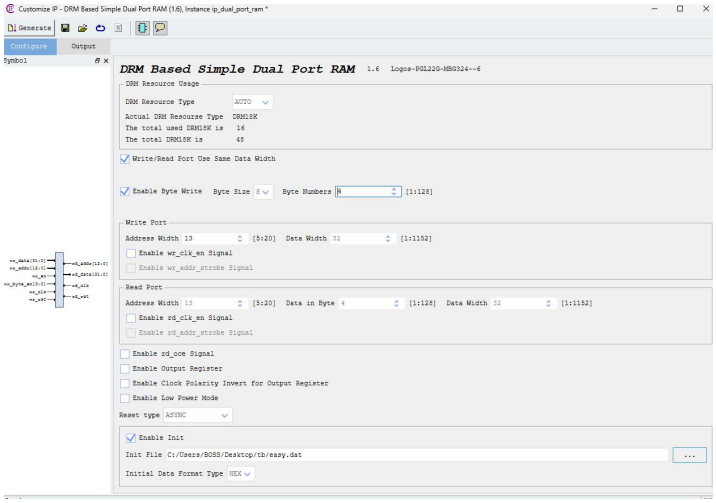


图 3-5 ip_dual_port_ram IP 核配置图

3.3.3 存储模块适配调整

替换 IP 核后，根据新 IP 核的接口特性，对原有代码逻辑进行适配和修改。具体修改过程如下：

（1）修改地址对齐逻辑

Ip_dual_port_ram 模块采用按字对齐的寻址方式，地址最低两位 addr[1:0]不参与存储单元选择。在原设计中，完整的地址信号直接用于内存访问操作。适配新 IP 核的接口标准要对地址信号进行裁剪，仅保留有效的高位部分，将最低两位舍弃，确保地址对齐后能够正确访问存储资源。

（3）适配写使能信号

原设计的存储模块是使用两个信号控制写操作：we_i 用于指示是否执行写操作，sel_i 用于选择需要写入的有效字节。紫光同创的 ip_dual_port_ram 模块将两者功能进行合并，由单一的 wen 信号完成控制。根据 IP 核的文档与功能仿真分析，最终确定 wen 信号的生成方式为：wen = ({4{we_i}} & sel_i)。

这样可以确保在 we_i 有效时，按照 sel_i 的位选择需要写入的字节，行为与原有设计保持一致，保证写操作的正确性。

（3）调整复位信号逻辑

Tinyriscv 设计中的复位信号采用低电平有效（resetn=0 时复位），但 ip_dual_port_ram IP 核则采用高电平有效复位逻辑（rst=1 表示复位有效）。因此需要对复位信号进行取反处理，转换为符合 IP 核规范的有效信号：rst = ~resetn。

通过对地址对齐逻辑、写使能信号和复位信号的适配修改，完成了存储模块 IP 核的移植工作。修改后的设计能在紫光同创 PGL22G FPGA 平台上正确映射存储资源，并保证了与原设计在功能行为上的一致性。完成 IP 核适配后，下一步把微控制器完整设计下载至 FPGA 实际硬件平台，开展原型验证，测试系统在硬件环境下的运行情况，进一步确认移植结果的正确性和稳定性。

3.3.4 移植后资源消耗率

经过上述适配修改后，tinyriscv 微控制器成功移植至紫光同创 PGL22G FPGA 平台，并在实际硬件环境中进行了综合与仿真验证。以下是移植后微控制器在开发板上的资源占用情况，统计如表 3.1 所示。

表 3.1 移植后开发板资源占用统计表

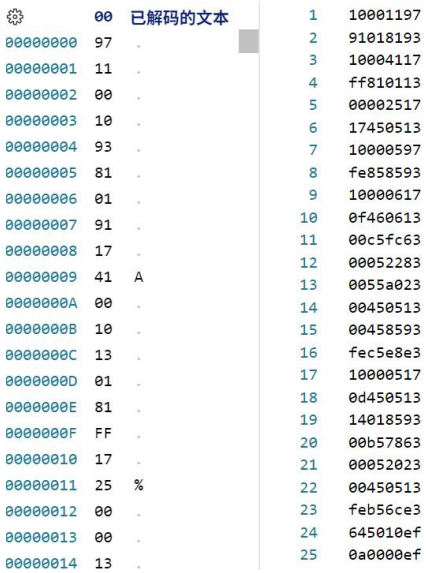
资源名称	消耗数量	总数量	消耗率%
LUT	4470	17536	25.34%
FF	3069	26304	11.66%
DRM	32	48	66.67%
IO	25	240	10.42%
APM	4	30	13.33%

Tinyriscv 微控制器在完成针对紫光同创 PGL22G FPGA 平台的移植与优化后，资源占用情况稳定并且符合预期，能够充分利用平台提供的逻辑资源，并在保证系统功能完整性的基础上，预留了足够的扩展空间，满足后续外设集成与性能优化的需求，验证了本设计的可移植性与实用性。

3.4 RTL 仿真运行 C 语言程序验证

在将编译生成的二进制文件正式下载至软核处理器并进行实际运行之前，通常需要通过仿真验证程序的正确性，以避免因代码逻辑错误或硬件实现问题导致运行异常。本设计采用了基于 ModelSim 的 RTL 级仿真方法，提前对 C 语言程序的运行过程进行验证。

将编译生成的二进制可执行文件（.bin 格式）通过 Python 脚本转换为.dat 格式文件。转换过程中，按照 ROM 模块指令存储的格式对二进制数据进行逐条拆分与重组，确保每一行数据能够正确对应处理器取指阶段的读取顺序。原始的.bin 文件是以单字节（Byte）形式顺序存储，每行对应一个数据单元。实际的 ROM 初始化过程中，需要按照每行 4 字节（Word）对齐的方式存储指令。所以使用 Python 脚本将.bin 文件重新格式化，转换为符合 ROM 加载要求的.dat 文件，确保指令在仿真或硬件上电时能够被正确识别和取用。Python 脚本完成文件转换的示意结果如图 3-6 所示。通过转换，可以方便地将编译好的程序加载到 FPGA 仿真环境，完成系统功能验证。



	00	已解码的文本
00000000	97	.
00000001	11	.
00000002	00	.
00000003	10	.
00000004	93	.
00000005	81	.
00000006	01	.
00000007	91	.
00000008	17	.
00000009	41	A
0000000A	00	.
0000000B	10	.
0000000C	13	.
0000000D	01	.
0000000E	81	.
0000000F	FF	.
00000010	17	.
00000011	25	%
00000012	00	.
00000013	00	.
00000014	13	.

```
1   10001197
2   91018193
3   10004117
4   ff810113
5   00002517
6   17450513
7   10000597
8   fe858593
9   10000617
10  0f460613
11  00c5fc63
12  00052283
13  0055a023
14  00450513
15  00458593
16  fec5e8e3
17  10000517
18  0d450513
19  14018593
20  00b57863
21  00052023
22  00450513
23  feb56ce3
24  645010ef
25  0a0000ef
```

图 3-6 二进制文件转 dat 文件示意图

将生成的.dat 文件在配置 ip_dual_port_ram IP 核时预加载到 ROM 存储空间，完成指令初始化。随后，通过提前编写的 Testbench 文件启动 RTL 仿真，模拟软核处理器的运行过程。在仿真时观察程序计数器（PC）的变化，按照指令宽度每次 PC 值应该递增 4，确保软核能够顺利读取指令。结合指令内容（INST）以及相关寄存器的读写，可以准确判断出运行的流程是否符合预期。

3.5 微控制器功能仿真验证

为验证 tinyriscv 微控制器在国产紫光同创 FPGA 平台下的功能正确性，设计了一段循环结构的测试程序，并在 ModelSim 仿真环境中对运行情况进行了分析。通过观察处理器执行指令的实际行为，确认移植后的软核架构是否能够在硬件环境中稳定运行。测试程序采用 C 语言编写，并通过交叉编译工具链生成机器码[22,23]。仿真时先对机器码进行了反汇编，得到对应的汇编指令序列，反汇编结果如图 3-7 所示。

```
Disassembly of section .data:

00000000 <.data>:
   0:   93000000                li      ra,0
   4:   13011000                li      sp,1
   8:   9301a000                li      gp,10
   c:   33822000                add     tp,ra,sp
  10:   93000100                mv      ra,sp
  14:   13010200                mv      sp,tp
  18:   9381f1ff                addi    gp,gp,-1
  1c:   e39801fe                bnez    gp,0xc
  20:   6f000000                j       0x20
```

图 3-7 机器码反汇编示意图

测试程序的指令结构比较简单，主要由寄存器赋值和循环控制两部分组成，能够有效验证处理器基本功能模块的正确性。程序中涉及的指令如表 3.2 所示。

表 3.2 汇编代码中涉及指令说明

指令	含义
li	将立即数赋值给指定寄存器
add	实现两个寄存器的加法运算，结果写回目标寄存器
mv	实现寄存器之间的数据传递，保持寄存器值不变的移动操作
addi	将一个寄存器与立即数相加，结果覆盖原寄存器
bnez	判断寄存器的值是否为零，不为零时跳转到指定地址形成循环控制结构

程序运行的过程能够覆盖整数运算、跳转指令、寄存器读写等处理器核心功能。仿真情况如图 3-8 所示。

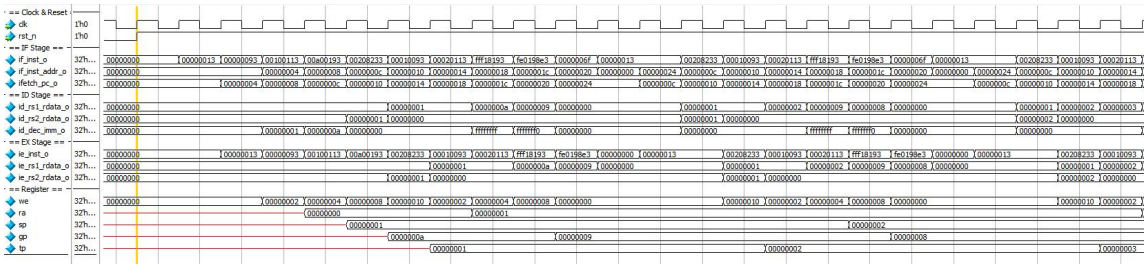


图 3-8 tinyriscv 功能仿真波形图

在仿真过程中选取处理器运行时有代表性的关键信号来进行观察，这些信号主要涉及时钟复位控制、指令执行流程以及寄存器组的读写状态等方面，以此来验证微控制器的功能完整性以及数据流的正确性。

首先是指令获取阶段即 IF Stage，其中 if_inst_o 呈现出当前取指阶段的指令具体内容，if_inst_addr_o 代表指令所处的地址，ifetch_pc_o 是程序计数器的当前数值，它可体现指令流的连续性以及跳转行为；接着是指令译码阶段也就是 ID Stage，id_rs1_rdata_o 和 id_rs2_rdata_o 分别对应着从寄存器堆里读取的源操作数，id_dec_imm_o 是指令解析后所提取的立即数，该阶段信号发生的变化，可对指令解码以及寄存器读操作的准确性进行验证；其次是执行阶段即 EX Stage，ie_inst_o 表示当前正处于执行阶段的指令，ie_rs1_rdata_o 与 ie_rs2_rdata_o 记录参与运算的操作数。结合执行结果能够判断算术运算、逻辑运算和跳转判断是否符合指令功能，验证运算单元的正确性。最后在寄存器写回阶段，we 信号指示是否执行寄存器写操作；ra、sp、gp、tp 分别对应寄存器组内不同寄存器的值变更过程，能够直观观察寄存器写操作的有效性，确保写回流程按预期完成。

通过波形图可以看到，处理器按照预设指令流程正常完成了指令的取指、译码、执行以及寄存器更新，寄存器状态变化与指令预期行为一致，说明 tinyriscv 软核在紫光同创 PGL22G FPGA 平台上已实现功能正确性验证，系统运行稳定。

3.6 PC 端与硬件交互工具 OpenOCD

在微控制器进行软硬件联合调试的过程里，调试工具的挑选对于开发效率以及测试准确性有着颇为重大的影响，OpenOCD 也就是 Open On-Chip Debugger，它是一款开源性质的嵌入式调试工具，可借助 JTAG 或者 SWD 接口达成处理器调试、程序下载、芯片烧录以及状态监测等功能，广泛地支持多种架构，像 RISC-V、ARM 等都在其支持范围内。在本设计当中，借助 OpenOCD 针对基于国产紫光 FPGA 平台所实现的 tinyriscv 软核来开展交互操作并进行程序下载[24]。

3.6.1 OpenOCD 工作原理

OpenOCD 运行在 PC 端，通过 USB 接口连接 JTAG 下载器（如 CMSIS、J-Link 等），下载器再与目标硬件建立通信链路，进而实现对处理器的调试与控制[25]。OpenOCD 工作框架如图 3-9 所示。

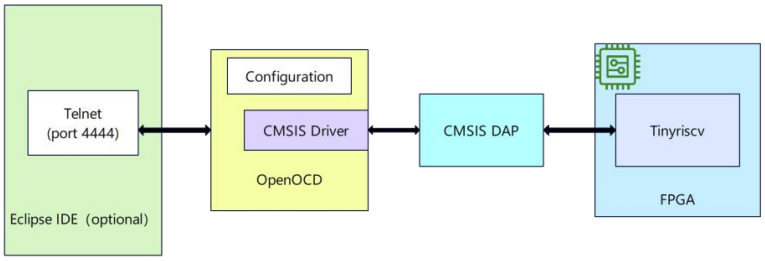


图 3-9 OpenOCD 工作框架图

在调试过程中，OpenOCD 要先通过配置文件预先定义的设备接口类型、通信速率和架构等信息。配置完成后可以通过 telnet 命令窗口与 OpenOCD 服务建立交互，通过这种方式，能够快速完成程序的验证与下载，提升软硬件联调的效率。

3.6.2 OpenOCD 环境配置及应用

运用 openocd 开展下载调试工作时，首先要做的是编写并且配置与之相应的配置文件，目的在于保证 PC 端调试工具可准确识别并连接目标设备，配置文件里，主要覆盖了如下信息，具体内容如表 3.3 所示。

表 3.3 OpenOCD 配置文件主要信息

指令	功能
adapter_khz	设置 JTAG 通信速率
interface	指定所使用的调试器类型
jtag newtap	配置目标设备的 JTAG 链路特性
target create	完成调试目标与 RISC-V 软核的绑定

完成配置文件后，通过 openocd -f tinyriscv.cfg 命令启动 OpenOCD 服务，OpenOCD 运行后会自动扫描 JTAG 链路，识别并建立与目标设备的通信连接。在另一个终端窗口通过 telnet localhost 4444 命令进入 telnet 的调试控制台。OpenOCD 打开命令行界面如图 3-10 所示。

```
C:\Users\BOSS\Desktop\Graduation\gitclone\tinyriscv\tools\openocd>openocd.exe -f tinyriscv.cfg
Open On-Chip Debugger 0.10.0+dev-00914-g4cb81306a-dirty (2020-01-28-21:35)
Licensed under GNU GPL v2
For bug reports, read
        http://openocd.org/doc/doxygen/bugs.html
Info : CMSIS-DAP: SWD  Supported
Info : CMSIS-DAP: JTAG Supported
Info : CMSIS-DAP: FW Version = 0255
Info : CMSIS-DAP: Serial# = 070000811a5a5bb50000000001ef46a3a5a5a5a597969908
Info : CMSIS-DAP: Interface Initialised (JTAG)
Info : SWCLK/TCK = 1 SWDIO/TMS = 1 TDI = 1 TDO = 0 nTRST = 0 nRESET = 1
Info : CMSIS-DAP: Interface ready
Info : clock speed 100 kHz
Info : cmsis-dap JTAG TLR_RESET
Info : cmsis-dap JTAG TLR_RESET
Info : JTAG tap: riscv.cpu tap/device found: 0x1e200a6f (mfg: 0x537 (Blue Liang), part: 0xe200, ver: 0x1)
Info : datacount=3 progbufsize=1
Info : Examined RISC-V core; found 1 harts
Info :  hart 0: XLEN=32, misa=0x0
Info : Listening on port 3333 for gdb connections
Info : Listening on port 6666 for tcl connections
Info : Listening on port 4444 for telnet connections
```

图 3-10 打开 OpenOCD 后的命令行界面

下面可通过 OpenOCD 提供的调试命令对软核进行操作。例如，程序下载时使用 load_image 命令可以将已编译完成的二进制文件下载到目标软核的内存中。引入 OpenOCD 有效提升了软核程序开发与测试的效率，简化了硬件集成与软件联调的流程，保证了系统开发过程的稳定性和可重复性。

3.7 跑分测试

为了评估微控制器的实际运算性能，采用 CoreMark 基准测试程序对系统运行效率进行了测试。跑分程序通常是由高级语言 C 或者 C++编写的一些规范的软件程序，可以让不同的处理器去运行这些跑分程序，从而得出相应的分数。作为一个衡量处理器性能好坏的指标，分数越高表示性能越好[26]。

CoreMark 的测试程序直接在 EEMBC 官网下载即可，程序里包含一些处理器经常用到的典型算法，例如链表操作、矩阵运算、状态机转换和循环冗余校验（CRC）等。因此，CoreMark 的跑分在嵌入式处理器领域中作为性能评价的重要参考指标。通过处理器每秒可循环执行 CoreMark 程序的次数作为标准，通过修改宏定义里的 ITERATIONS 参数来修改循环执行的次数，计算出来每秒测试程序循环的次数，再除以处理器的主频即可得出 CoreMark 程序跑分，即：CoreMark/MHz。

在基于 FPGA 的 tinyriscv 软核平台上运行 CoreMark 程序和下载步骤与上述相同。最后将运行完的 CoreMark 程序的信息通过串口打印到 PC 端的串口显示终端上，如图 3-11 所示，执行 CoreMark 程序循环次数为 10000 次，每秒执行程序循环次数 84 次，tinyriscv 在 FPGA 上是以 50MHz 的主频运行 CoreMark 程序，即得分为 1.68，接近 ARM Cortex-M0 的运行跑分程序的得分。

```
RX: 2K performance run parameters for coremark.
CoreMark Size    : 666
Total ticks      : 1637193147
Total time (secs): 118
Iterations/Sec   : 84
Iterations       : 10000
Compiler version : GCC8.2.0
Compiler flags   : -O2 -fno-common -funroll-loops -finline-functions --param max-
inline-insns-auto=20 -falign-functions=4 -falign-jumps=4 -falign-loops=4
Memory location  : STATIC
seedcrc          : 0xe9f5
[0]crclist       : 0xe714
[0]crcmatrix     : 0x1fd7
[0]crcstate      : 0x8e3a
[0]crcfinal      : 0x988c
Correct operation validated. See readme.txt for run and reporting rules.
```

图 3-11 CoreMark 跑分结果图

3.8 系统整体架构

至此整个开发平台及工具已经搭建完成，覆盖了硬件设计、软件开发与调试验证的全流程，实现了从硬件实现的软件开发的闭环。整体可分为两大部分，主要包含硬件和软件两部分。

硬件部分主要包括硬件描述语言 Verilog 实现的 tinyriscv 软核、紫光同创 PGL22G FPGA 开发板和 JTAG 调试器，负责 RISC-V 指令集外设资源的管理，提供了底层硬件计算资源。

软件部分主要是依托配套的软件开发平台 SDK 和相关的软件工具链，实现了程序编写编译、烧录以及仿真调试功能，与硬件平台无缝对接。

通过软硬件协同开发，开发平台能够支撑 tinyriscv 软核的验证、功能扩展及优化，系统整体框图如图 3-12 所示。

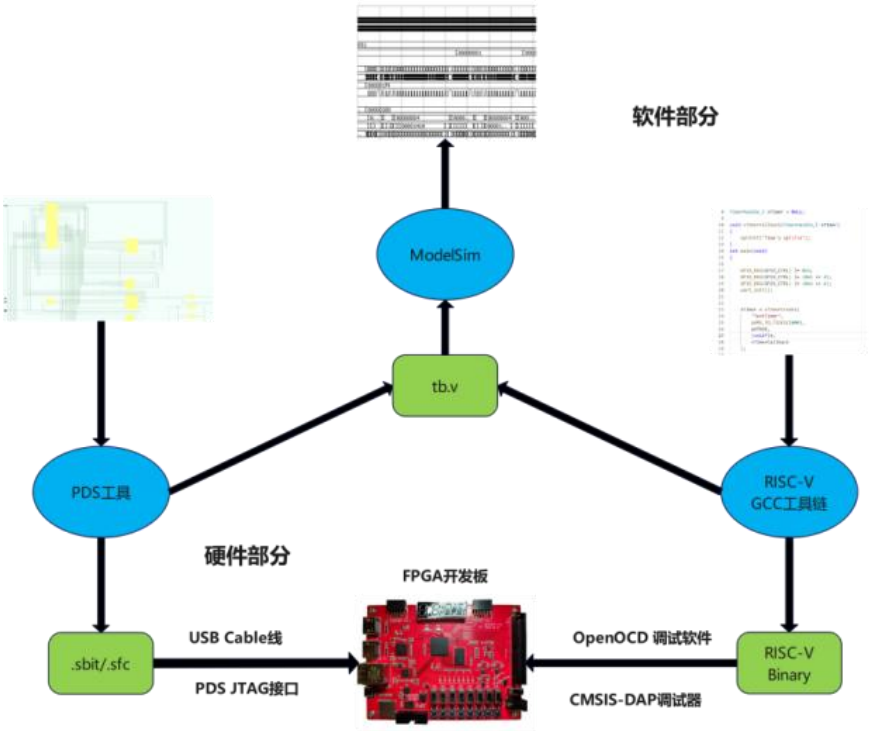


图 3-12 系统总体示意图

3.9 本章小结

本章围绕基于国产紫光平台的 tinyriscv 微控制器设计与移植进行了系统性描述，首先介绍了微控制器的整体架构及各核心模块的设计思路，分析了处理器在 IF、ID、EX 各流水线阶段的实现方法。然后结合紫光 FPGA 开发板的资源特点，完成了 tinyriscv 的适配与移植，针对存储模块进行了调整优化，并通过资源消耗率评估了移植效果。通过 RTL 仿真与功能仿真验证了处理器的正确性，同时结合 OpenOCD 工具实现了 PC 端与硬件平台的交互与调试，进一步通过跑分测试评估了系统性能。

4 微控制器外设系统的设计与实现

完成处理器核心架构的设计后，为确保软核系统能够实现基本的输入输出控制与外部设备交互，本项目针对基础外设功能模块，完成了外围设备的设计、集成与功能验证[27,28]。模块名称及具体地址空间分配如表 4.1 所示。本章将围绕各外设模块的设计原理与验证过程进行详细阐述，并通过功能仿真波形以及板级测试结果，证明设计方案的可行性和正确性。

表 4.1 各外设模块地址空间分布表

模块名称	地址空间
SPI	0x5000_0000--0x5FFF_FFFF
GPIO	0x4000_0000--0x4FFF_FFFF
UART	0x3000_0000--0x3FFF_FFFF
TIMER	0x2000_0000--0x2FFF_FFFF
RAM	0x1000_0000--0x1FFF_FFFF
ROM	0x0000_0000--0x0FFF_FFFF

4.1 RAM 和 ROM

在此次设计里，RAM 也就是数据存储器以及 ROM 即指令存储器模块，都是依据 FPGA 厂商所提供的 ip_dual_port_ram IP 核来达成的。存储器运用双端口读写架构，以此契合处理器对于指令以及数据的高效访问要求，这种架构的设计可充分运用双端口的特性，使得数据与指令的读写操作可在不同端口同时开展，这样便提高了存储器的访问效率，减少了处理器的等待时长，提升了整个系统的运行速率。

虽说 RAM 和 ROM 是基于一样的 IP 核结构实现的，不过在系统里的作用以及使用方式却不一样。RAM 主要承担存储程序运行时产生的动态数据，像是变量、中间计算结果以及缓存信息等，因为数据会随着程序的执行持续更新，RAM 要支持读写操作，保证处理器可及时读取最新数据，并且将计算结果正确写入内存。ROM 用于存储程序指令，主要支持读操作，程序指令在编译阶段就已固定，运行过程中无需修改，指令固定存储还可以提高系统的安全性，防止外部干扰或者代码篡改造成的运行异常，在实际应用中，嵌入式设备大多时候采用 ROM 存储固化的程序代码，保证系统在重启后可恢复稳定运行。

前文工程移植部分已经说明了相关的 IP 核配置方法，这里就不再重复讲述，通过合理划分 RAM 和 ROM 的功能，系统契合了处理器运行时的访问需求，还兼顾了系统的稳定性与安全性，为整个系统的稳定性提供了硬件保障。

4.2 TIMER 模块

TIMER 定时器模块是系统里用于时间控制的关键部件，其主要承担着任务定时以及中断触发等功能。在嵌入式系统当中，定时器大多时候被用于周期性任务的调度以及硬

件中断源的生成，定时器的精确性与稳定性会直接对系统的运行产生影响。

本设计的 TIMER 模块可让开发者依据需求设置计数的时间，之后借助中断机制告知处理器完成当前的定时任务。定时器采用递增计数的工作模式，开发者借助配置寄存器来给计数器加载初始值，计数器配置妥当后，会依据系统的时钟开始递增计数，直至计数达到配置的初始值，此时模块会输出一个中断信号，用以提醒主控处理器当前定时任务已完成。

TIMER 模块结构简单，并且资源占用小，比较适合在资源受限的嵌入式平台上面部署。通过灵活的配置定时器寄存器和中断信号，该模块可以满足对系统时间控制的基本功能，以此提高系统运行的效率和稳定性。

4.2.1 TIMER 寄存器设计

为实现对 TIMER 模块的配置与状态监测，设计了 3 个功能明确的寄存器，分别用于控制、计数初值设置和状态反馈。各寄存器的地址及功能如表 4.2 所示。

表 4.2 TIMER 模块寄存器功能表

寄存器名称	地址偏移	功能介绍
TIMER0_CTRL	0x00	控制寄存器：配置定时器启动、中断使能及工作模式
TIMER0_COUNT	0X04	初值寄存器：设置定时器的初始计数值
TIMER0_STATUS	0X08	状态寄存器：反馈定时器当前状态

各寄存器具体功能说明如下：

（1）TIMER_CTRL:bit[0]：定时器使能位，1 表示启动定时器，0 表示关闭；bit[1]：中断使能位，1 表示允许在计数完成时触发中断，0 表示禁止中断；bit[2]：工作模式选择位，1 表示循环模式，计数完成后自动重新加载初值开始下一轮计数；0 表示单次模式，计数完成后停止。

（2）TIMER_COUNT:bit[31:0]：定时器初始计数值。写入该寄存器后，计数器将在下次启动时加载此值，并开始从零递增计数，直至到达此数值触发事件。

（3）TIMER_STATUS:bit[0]：状态标志位。1 表示计数器当前处于运行状态；0 表示计数完成，等待重载或新一次启动。

4.2.2 TIMER 功能测试与验证

为了验证 TIMER 模块的功能正确性与稳定性，设计了一套基于中断机制的周期性定时测试方案。测试程序通过配置 TIMER 模块的寄存器初值，实现定时器按照固定时间触发一次中断，并在中断服务触发后通过程序中 count 进行累加自增，从而实现对模块的定时精度与中断响应功能的综合验证。

测试程序的基本流程如下：程序运行后，先通过向 TIMER0_COUNT 寄存器写入设定的初值 500，配置定时器的计时周期；并且通过设置 TIMER0_CTRL 寄存器，启动定时器并使能中断功能。完成设置后，程序进入循环等待中断计数变量 count 的变化。当计数值达到设定次数（本测试设定为 2 次）时，代表定时器已按预期稳定运行，主程序

通过调用 set_test_pass()函数，将测试标志寄存器进行拉高，表示测试通过。测试代码如下所示：

```
int main(){
count = 0;
  TIMER0_REG(TIMER0_VALUE) = 500;
  TIMER0_REG(TIMER0_CTRL) = 0x07;
  while (1) {
    if (count == 2) {
      TIMER0_REG(TIMER0_CTRL) = 0x00;  count = 0;
      set_test_pass();break;}}
void timer0_irq_handler(){
  TIMER0_REG(TIMER0_CTRL) |= (1 << 2) | (1 << 0);
  count++;}
```

在测试过程中，TIMER 模块自 0 开始递增计数，当计数值等于预设目标值 500 时，自动置位中断标志，并根据 TIMER0_CTRL 的配置，向处理器发出中断请求，CPU 根据配置跳转执行 timer0_irq_handler 中断服务程序。进入中断后，程序首先对 TIMER0_CTRL 寄存器进行写操作，清除中断标志，并重新启动定时器，确保进入下一轮计数循环。同时通过自增 count 变量，记录累计中断触发次数。

如图 4-2 所示，TIMER 模块能够稳定完成从 0 递增计数，准确触发中断信号，符合循环模式下的预期行为。观察测试信号 int_sig_r 的电平变化，可看出中断是否按预期正常触发。当 int_sig_r 信号被拉高时，表示 TIMER 已完成一次完整的计数周期并成功发出中断请求中断累计触发 2 次后，主程序通过 set_test_pass()函数将寄存器第 27 位（S10）拉高，表示本次功能测试顺利通过，此时结果证明了 TIMER 模块具备稳定的计时和触发中断功能，满足系统设计要求。

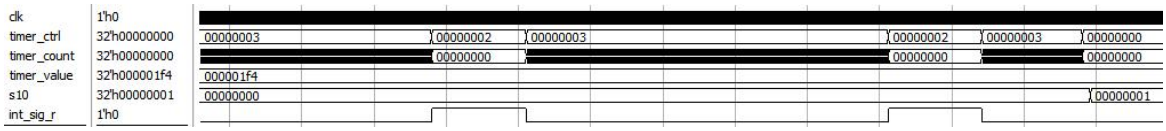


图 4-1 TIMER 功能仿真波形图

4.3 UART 模块

UART（Universal Asynchronous Receiver/Transmitter，通用异步收发器）是一种在微控制器系统中广泛应用的异步串行通信协议，主要用于芯片与外设或上位机之间的双向数据传输。UART 通信通常采用异步传输的方式，因此不需要额外的时钟信号仅使用两根线就可以完成通信，TX 同于发送，RX 用于接收。通信线路结构简单，能够有效减少硬件接口资源消耗。

在本设计中，UART 模块实现了软核与 PC 端之间的串行数据交互，支持标准的 8-N-1 传输格式（1 位起始位、8 位数据位、无奇偶校验和 1 位停止位）和 8-N-2 传输格式，可

以适配不同场景下的通信需求。模块内部设定了可编程的波特率分频器，因此开发者可以根据系统所需要的通信速率进行灵活修改，提高了通信速率的稳定性和移植性。

4.3.1 UART 寄存器设计

UART 模块主要由发送状态机、接收状态机、波特率分频器、数据寄存器以及控制寄存器组成。发送部分通过状态机控制，实现按帧结构顺序自动发送起始位、数据位和停止位，确保发送完整的帧结构，发送状态机的状态跳转如图 4-3 所示。数据每次写入到 TXDATA 寄存器后，根据设定的波特率自动发送数据。接收部分同样采用状态机逻辑，对输入信号进行采样，接收状态机在检测到起始位后，按照设定波特率采样数据位，最后确认停止位，确保异步传输数据的稳定性和正确性[29]。

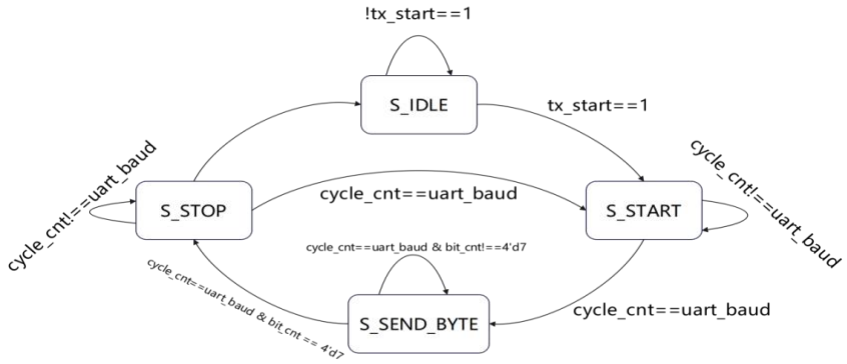


图 4-2 发送状态机跳转图

UART 模块的寄存器功能结构如表 4.3 所示。

表 4.3 UART 模块寄存器功能表

寄存器名称	地址偏移	功能介绍
UART_CTRL	0x00	控制寄存器:bit[0]: TX 使能;bit[1]: RX 使能
UART_STATUS	0x04	状态寄存器，指示发送忙碌、接收完成等状态
UART_BAUD	0x08	波特率分频寄存器，设置通信速率
UART_TXDATA	0x0c	发送数据寄存器，写入待发送的数据
UART_RADATA	0x10	接收数据寄存器，读取接收的数据

4.3.2 UART 功能测试与验证

验证 UART 模块的基本功能，编写了一个循环发送测试程序，测试程序通过不断向发送寄存器写入数据，检验 UART 模块的稳定性和正确性。测试代码如下所示：

```
int main(){
  uart_init();
  int i = 0;
  while(1){
    uart_putc(i); i++;}}
```

在测试过程中，程序首先调用 uart_init 初始化配置，包括波特率设置与收发使能。随后进入循环，持续向 uart_putc 函数传递递增的计数值，观察 UART 模块的数据传输过程。从仿真波形图可以观察到，整个传输过程的逻辑关系符合设计预期，具体流程如下：

在发送过程中，状态机会按照逻辑顺序控制 TX 引脚输出。先进入起始位阶段，TX 输出低电平，接着状态机根据 bit 计数器从 0 到 7 依次将数据从最低位开始逐位发送至 TX 引脚直到 8 位数据传输完毕。当检测到停止位是 TX 引脚会拉高，等待下一次的传输。如图 4-4 所示，仿真波形可以观察到 bit_cnt 信号从 0 递增到 8 的过程，完整对应数据帧的传输过程。tx_pin 信号的变化与实际 UART 标准帧结构完全吻合。仿真结果表明，UART 模块能够正确按照协议完成数据打包与传输，uart_status 寄存器能够准确反映发送状态，确保软件和硬件之间的读写逻辑同步一致。

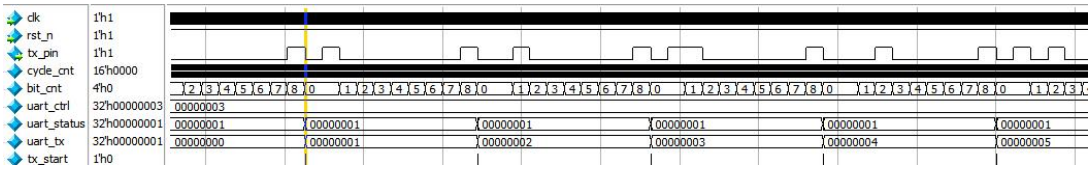


图 4-3 UART 功能仿真波形图

除了仿真波形验证外，还通过实际串口进行物理测试，结果如图 4-5 所示。测试时，软核通过 UART 模块向 PC 端持续发送数据，PC 端使用 XCOM 串口助手进行接收，结果表明数据传输过程中未出现丢包或数据错误，数据能够准确无误地接收并显示，进一步验证了 UART 模块的功能正确性。

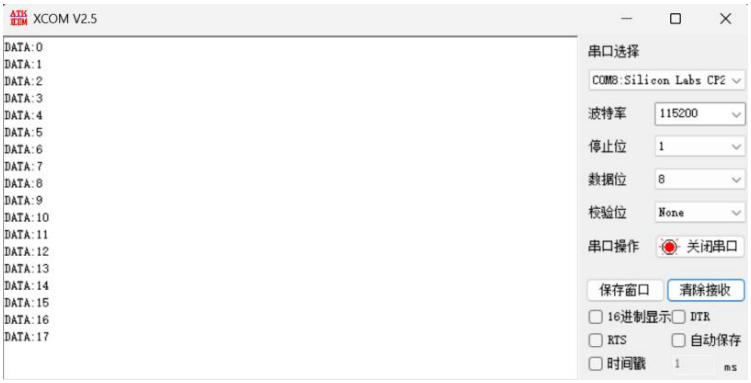


图 4-4 UART 串口测试截图

4.4 SPI 模块

SPI 即串行外设接口，属于同步串行通信协议，常被用于微控制器与外设之间的数据传输工作。SPI 通信采用主从结构模式，数据交换依靠时钟同步得以完成，其传输速率较快而且稳定，硬件实现过程较为简单，故而被广泛应用于如存储器、液晶屏等外设控制方面。

本设计里的 SPI 模块采用的是主机模式，契合标准四线制接口。通信信号涉及时钟线 SCLK、主机输出从机输入的 MOSI、主机输入从机输出的 MISO 以及片选信号 SS，该模块支持对通信参数进行灵活配置，开发者可依据外设需求来设置时钟的极性 CPOL

及时钟相位 CPHA。在传输数据时，模块内部的状态机能自动控制数据的输入与输出，同时同步更新状态标志以指示传输的起始与结束，以此保证主从设备交互数据的完整性。

4.4.1 SPI 寄存器设计

SPI 模块包含 3 个核心寄存器，分别用于功能控制、数据传输和状态反馈，具体的寄存器地址及功能概览如表 4.4 所示。

表 4.4 SPI 模块寄存器功能表

寄存器名称	地址偏移	功能介绍
SPI_CTRL	0x20	控制寄存器：用于配置 SPI 的工作参数
SPI_DATA	0x24	数据寄存器：用于存放待发送或接收的数据
SPI_STATUS	0x28	状态寄存器：用于反馈 SPI 模块当前的工作状态

寄存器功能说明：

（1）SPI_CTRL:bit[0]用于启动或停止传输（1：使能，0：关闭）；bit[1]配置时钟极性 CPOL；bit[2]配置时钟相位 CPHA；bit[3]控制片选信号，1 表示选中从设备，0 表示释放；bit[15:8]用于设置时钟分频系数，从而调节 SPI SCLK 的频率，满足不同外设的通信速率要求。

（2）SPI_DATA:bit[7:0]为有效数据位，软件通过向该寄存器写入数据启动发送，或在接收完成后从该寄存器读取数据。

（3）SPI_STATUS:bit[0]表示传输状态，1 为“忙碌”，表示数据正在传输；0 为“空闲”，表示传输已完成，可以进行下一次操作。

4.4.2 SPI 功能测试与验证

为了验证 SPI 模块的功能正确性，设计了一个简单的循环数据发送测试程序，代码如下所示：

```
int main(){
  spi_init();
  spi_set_ss(0);
  while(1){spi_write_byte(0xAA);busy_wait(1);
    spi_write_byte(0x11);busy_wait(1);
    spi_write_byte(0x22);busy_wait(1);
    spi_write_byte(0x55);busy_wait(1);
    spi_write_byte(0xFF);busy_wait(1);
}}
```

测试时，spi_write_byte 函数会将待传输的数据写入到 SPI_DATA 寄存器，并通过配置 SPI_CTRL 寄存器启动传输。状态机识别到启动命令后会将 SPI_STATUS 寄存器的 busy 标志置为 1，表示传输开始，避免在传输过程中再次写入数据。

仿真波形结果如图 4-6 所示，SPI 模块能够稳定地将数据按照设定顺序逐位输出至 MOSI 信号线，信号的变化符合 CPOL 和 CPHA 参数的配置，数据帧结构正确，传输行为符合 SPI 协议的规范。在每次数据传输完成后，状态机会自动将 busy 标志清零，表示传输完成，模块会进入空闲状态，等待下一次数据的写入。并且数据总线在传输结束后的空闲期进入高阻态（Z 状态），避免了总线上的干扰，提升了总线的稳定性。通过循环测试验证，确认 SPI 模块的状态切换稳定，传输数据完整，能够正确完成主从设备之间的通信任务。

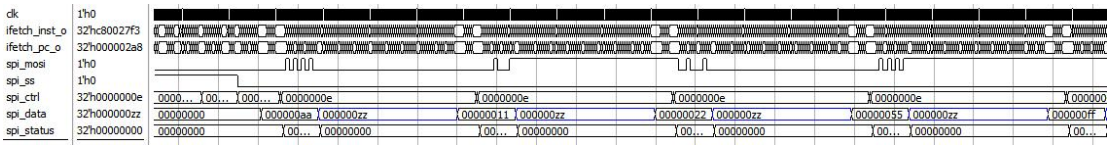


图 4-5 SPI 功能仿真波形图

为了进一步验证 SPI 模块的实际应用效果，设计了一个基于 SPI 接口驱动显示屏的测试实验。软核通过 SPI 向 LCD 显示屏发送显示指令，完成屏幕的初始化显示。如图 4-7 所示。

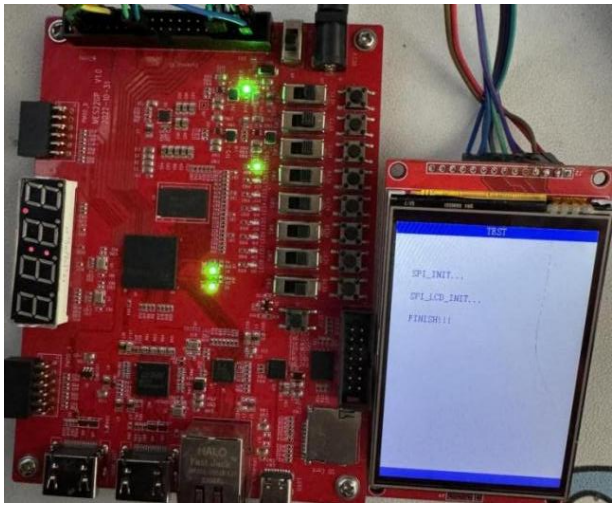


图 4-6 SPI 驱动 LCD 显示屏测试图

4.5 GPIO 模块

GPIO 也就是通用输入输出模块，作为嵌入式系统里极为常用的基础外设，主要发挥着处理器与外部设备间数字信号输入及输出控制的作用。借助 GPIO 模块可采集外设的状态信息，还可向外设输出控制信号，达成简单又可靠的数字信号通信，在硬件设计过程中，GPIO 模块大多时候被应用于按键检测、LED 指示等场景，其结构简单而且十分灵活。

本设计所采用的 GPIO 模块运用的是寄存器映射方式，处理器借助访问指定地址，实现对 GPIO 相应引脚的配置以及读写操作，该模块内部囊括数据输入、数据输出以及方向控制寄存器，可契合设计里基本的输入输出需求。

4.5.1 寄存器设计

GPIO 模块通过两个寄存器控制引脚的模式和数据,具体的寄存器地址及功能概览如表 4.5 所示。

表 4.5 GPIO 模块寄存器功能表

寄存器名称	地址偏移	功能介绍
GPIO_CTRL	0x00	用于设置各 GPIO 引脚的工作模式
GPIO_DATA	0x04	用于引脚设定或读取的数据

GPIO_CTRL（控制寄存器）：每两位控制一个 GPIO 引脚，具体含义如下：

-00 高阻态；-01 输出模式；-10 输入模式；-11 保留。

GPIO_DATA（数据寄存器）：根据引脚配置的模式有不同功能。当设置为输出模式时，对应位可写入值，直接决定输出电平；当设置为输入模式时，对应位反映引脚的输入电平，供处理器读取。

通过寄存器配置，开发者可以灵活设置引脚工作模式，并实现对外设的控制和监测，满足多样化的应用场景需求。

4.5.2 功能测试与验证

为了验证 GPIO 模块的功能正确性，设计了一个简单的循环测试程序，采用 gpio0 引脚作为测试对象。测试代码如下：

```
int main() {
  GPIO_REG(GPIO_CTRL) |= 0x1;
  while (1) {
    GPIO_REG(GPIO_DATA) |= 0x1;
    GPIO_REG(GPIO_DATA) &= ~0x1;
}}
```

程序运行后，首先向 GPIO_CTRL 寄存器写入初始化配置的值，将 gpio0 设为输出模式，随后在程序中进入了死循环，反复对 GPIO_DATA 写入高电平和低电平，驱动 gpio0 的翻转，通过这种方式可以直接观测到 GPIO 模块的输出是否正常。

仿真波形如图 4-8 所示，处理器能够正确控制 GPIO_DATA 寄存器，完成引脚电平的翻转操作，且输出逻辑与软件行为一致。能够满足系统对外设的交互需求。

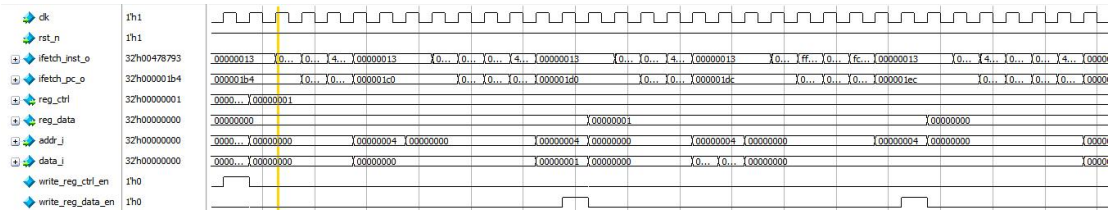


图 4-7 GPIO 功能仿真波形图

4.6 本章小结

本章对微控制器中的外围设备包括 GPIO、UART、SPI、TIMER、RAM 和 ROM 模块的原理进行了介绍，对各个模块的寄存器的主要功能以及设计进行了概述。此外，还通过编写不同 C 语言程序代码并将其编译后下载到微控制器运行，查看功能仿真波形图，完成了对各模块功能的测试。

5 基于 tinyriscv 的 RTOS 移植

5.1 RTOS 的简介

FreeRTOS（Free Real-Time Operating System）是一款针对嵌入式系统的轻量级实时操作系统，它由英国的 Real Time Engineers Ltd.进行开发与维护，遵循 MIT 开源协议，有不错的可移植性以及易裁剪性。FreeRTOS 是专门为资源受限的嵌入式平台设计的，可给微处理器提供如任务调度、中断管理、时间管理、同步机制等基础操作系统功能，方便搭建稳定的多任务应用框架。和传统裸机开发方式相比较，FreeRTOS 借助调度器与任务机制，提升了程序结构的模块化程度，提高了系统的并发处理能力以及稳定性，简化了复杂场景下的开发工作。该系统支持抢占式和协作式这两种调度策略，开发者可依据实际需求设置任务优先级，保证关键任务可以及时响应，契合系统对实时性的要求[30]。

FreeRTOS 采用简化内核设计，其核心代码体积较小，一般只占用数 KB 的存储空间，适合嵌入式系统部署，FreeRTOS 已经支持多种主流处理器架构，像 ARM Cortex-M、RISC-V、MIPS、PowerPC 以及 AVR 等。内核功能划分清晰，主要有任务调度模块、时间管理模块、内存管理模块以及同步通信模块，便于依据硬件资源裁剪功能，

FreeRTOS 的硬件抽象层结构简单，移植过程完成启动文件、中断向量表配置以及任务切换底层的相关代码就能完成适配，提高了系统的开发效率，本设计把 FreeRTOS 成功移植到国产 FPGA 平台的 tinyriscv 软核处理器上，实现了基本的多任务调度功能与中断响应机制。引入 FreeRTOS 后，软核处理器可运行复杂的多任务嵌入式程序，还可以验证处理器在任务切换、上下文保护以及中断处理等功能完整性与可靠性。

5.2 FreeRTOS 的移植

FreeRTOS 项目的源码托管在 GitHub 上，从网站上克隆该项目的 v10.3.1 版本的代码，目录结构包含内核源文件、移植层代码和平台适配文件。不同架构平台的适配文件存放在 portable 目录下。具体的 FreeRTOS 目录结构如下：

```
FreeRTOS
    ├Source/      #FreeRTOS 内核源码
        ├include/     #FreeRTOS 的头文件
        ├portable/          #移植层
            ├MemMang/            #内存管理模块
            ├RISC-V/                    #RISC-V 特定适配代码
                ├freertos_risc_v_chip_specific_extensions.h
                ├port.c
                ├portASM.S
                ├portmacro
    ├Demo/       #示例工程
```

本设计围绕 RISC-V 架构展开，运用 GCC 编译工具链，在移植进程里着重对 port.c、portmacro.h 以及 portASM.S 这三个核心文件给予修改和配置。

（1）port.c 文件存有 FreeRTOS 内核调度机制以 C 语言形式呈现的实现代码，涉及堆栈初始化、任务调度器启动、Tick 中断处理等功能，以此保证系统达成稳定的上下文切换与调度。

（2）portmacro.h 文件界定了系统运行所需的基本数据类型、和内核调度相关的关键宏以及与架构相关的内存访问操作，对底层硬件的差异性起到屏蔽作用，提升了系统代码的可移植性。

（3）portASM.S 文件运用汇编语言实现，包含任务切换过程中的上下文保存与恢复、中断处理的入口跳转以及调度器启动等底层逻辑，保证多任务运行顺畅。

在使用 FreeRTOS 时可以依据具体的处理器微架构进行配置，在系统功能配置文件 FreeRTOSConfig.h 中主要包括一些 define 语句的宏定义声明，比如时钟滴答频率、最小堆栈大小、任务最大优先级、内存分配方式等配置，钩子函数的相关配置，软件定时器的相关配置以及设置一些函数的使能等。

经过上述文件的配置与修改，最终通过编写 Makefile 文件，将 FreeRTOS 内核源文件、平台适配代码、硬件配置文件统一编译链接，结合主程序完成系统的功能测试，重点验证任务调度的正确性、任务切换的时序精度以及中断处理机制的稳定性。

5.3 仿真验证与上板测试

在完成 FreeRTOS 的移植后，设计了一个简易的任务调度验证方案，结合仿真环境对系统运行效果进行测试。测试程序中创建了两个任务，分别用于模拟数据的发送和接收。发送任务按照设定的时间间隔向消息队列中写入固定数值，接收任务从队列中取出数据，并在接收到预期值时调用 GPIO 控制函数，切换 GPIO0 的输出状态。

测试程序编译完成后，将二进制文件加载至 ROM 存储空间，启动仿真。测试重点监测以下信号：

（1）程序计数器 ifetch_pc_o 和指令内容 ifetch_inst_o，验证程序执行是否正确；

（2）GPIO 相关信号 gpio_ctrl 和 gpio_data，确认外设状态变化；

（3）定时器信号 timer_ctrl、timer_count、timer_value、int_sig_r，确认时间片中断的准确性；

（4）CSR 寄存器 mtvec、mcause、mepc 和 mstatus，验证中断入口和返回逻辑的正确性。

仿真结果如图 5-1 所示，TIMER 模块在系统初始化后按设定周期开始计数。当计数达到 500 时，自动触发中断。CPU 进入中断服务程序，完成相关处理后返回主程序。经过连续 20 次中断触发，接收任务按照逻辑判断条件顺利调用 GPIO 控制函数，GPIO0 状态被翻转。任务切换与外设控制流程均符合设计预期，证明 FreeRTOS 的移植正确。

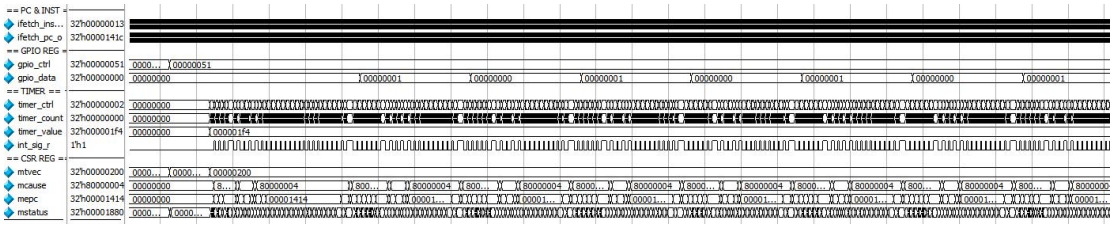


图 5-1 FreeRTOS 移植后仿真波形图

在仿真验证通过后，将编译生成的二进制程序文件烧录至紫光同创 PGL22G FPGA 开发板中，进一步开展实机测试。在测试中，系统通过串口输出方式打印当前时钟信息，并同步控制 GPIO0 的信号翻转。

如图 5-2 所示，串口终端能够按预期稳定输出时钟信息，且每隔一秒更新一次，GPIO0 的信号也同步发生翻转。FreeRTOS 在 tinyriscv 软核平台上的移植方案在实际硬件平台上能够实现稳定运行，任务调度、中断响应和外设控制功能均符合预期设计要求。

图 5-2 上板运行 FreeRTOS 串口输出时钟

5.4 本章小结

本章介绍了 FressRTOS 操作系统的原理以及 FressRTOS 的移植过程，然后对 RISC-V 微控制器平台进行了 FreeRTOS 的移植，通过仿真验证与上板测试验证了操作系统移植的正确性。

6 结论与展望

6.1 结论

本文借助国产 FPGA 平台达成了 tinyriscv 软核的实现工作，该设计运用 RISC-V 开源指令集架构，依照软核微处理器的特性，达成了从核心架构构建直至外设模块开发，到操作系统移植的一整套流程，搭建起一个适配嵌入式系统的软硬件协同验证平台。

设计进程里，先是剖析了 RISC-V 指令集架构的设计思路以及技术背景，明晰了其有开放、模块化以及灵活可裁剪的优势，这种优势特别契合资源受限且对自定义能力要求较高的嵌入式系统。紧接着，结合国产紫光同创 FPGA 平台的硬件特性，完成了 tinyriscv 软核的适配与优化，针对存储模块的结构设计、地址映射方案以及总线接口规范，展开了细致的分析与调整，以保证软核在实际平台上的功能完整性和稳定性。

在外设模块方面，完成了存储、TIMER、GPIO、UART、SPI 等基础功能模块，明确了各模块的寄存器映射，并且实现了硬件信号的正确交互。经过功能仿真以及开发板实机测试，验证了各外设模块的稳定性与可靠性，保证其符合软核总线访问请求，可契合系统运行的基本需求，在开发流程中，采用 ModelSim 进行 RTL 级功能仿真，以此保证硬件逻辑的正确性，引入 OpenOCD 工具，完成程序的下载与调试，保障硬件平台可正常加载和执行应用程序。

在此基础之上，完成了轻量级实时操作系统 FreeRTOS 的移植工作，着重解决了中断管理、堆栈初始化以及与软核架构的适配问题，实现了系统稳定运行，依靠仿真测试以及实机测试，验证了 FreeRTOS 在 tinyriscv 软核平台上的功能正确性，提升了系统的多任务调度能力和可扩展性，契合了基本的实际应用需求。

综上所述，本设计完成了基于国产 FPGA 的 tinyriscv 软核系统实现，验证了 RISC-V 架构在嵌入式领域的可行性，还提供了一套完整的软硬件协同开发流程，设计采用模块化架构，方便后续拓展新的外设功能，也为其他操作系统的移植奠定了基础。

6.2 展望

（1）目前外设模块的种类较为有限，仅实现了最基础的通用输入输出、定时器、串口通信、SPI 接口等功能，未涉及复杂的通信协议及高级外设（如 PWM，HDMI 等）。后续根据实际应用需求，拓展更多种类的外设，提升适用范围。

（2）系统性能优化不充分，现阶段软核设计主要以功能正确性验证为主，未在功耗控制、任务调度效率、缓存优化等方面深入研究，特别是在多任务并发和资源争用条件下，系统的运行稳定性与响应速度有优化空间。

（3）指令集扩展与硬件加速功能未涉及，本设计基于 RV32I 基本指令集，尚未引入 RISC-V 扩展指令（如 RV32A 原子指令集、向量扩展等）。未来可针对应用需求，设计自定义扩展指令或硬件加速模块，提升特定任务（如加密运算、信号处理）的计算效率。

参考文献

[1] 刘小罗,林洪怡,刘盼.RISC-V指令集架构及其应用综述[J].中国集成电路,2025,34(03):16-20+49.

[2] Waterman E A, Lee Y, Patterson D A.The RISC-VInstruction Set Manual, Volume I: Base User-Level ISA [R].Vol. 1. 2011.

[3] 中国科学院文献情报中心咨询服务部信息学科情报团队,李海英,闫亚飞,等.趋势观察:RISC-V架构芯片研究进展[J].中国科学院院刊,2022,37(01):126-129.DOI:10.16418/j.issn.1000-3045.20211215001.

[4] 王坤,王伟,付相为.基于FPGA的高清压缩视频光纤传输系统设计[J].信息技术,2025,(04):1-7+15.DOI:10.13274/j.cnki.hdzj.2025.04.001.

[5] OMDIA. RISC-V Global Market-2024 Analysis[EB/OL]. Omdia, 2024[2025-04-29]. https://omdia.tech.informa.com/om121430/risc-v-global-market--2024-analysis.

[6] 李孟晗.面向物联网应用的RISC-V架构SoC设计与实现[D].哈尔滨工业大学,2023.DOI:10.27061/d.cnki.ghgdu.2023.002408.

[7] 王弘毅.助力国产车规级芯片走上自主可控之路[N].安徽日报,2025-03-24(001).DOI:10.27996/n.cnki.nahrb.2025.001489.

[8] 平头哥发布全新 RISC-V 处理器[J]. 中国集成电路, 2021, 30(06): 21.

[9] 赵坤.基于RISC-V架构的SoC设计与实现[D].海南大学,2021.DOI:10.27073/d.cnki.ghadu.2021.000238.

[10] 全球首颗智能穿戴领域人工智能芯片发布[J].智能城市,2019,5(10):191.

[11] Lun T L ,Khairi M I ,Khalid A . Design and implementation of secure boot architecture on RISC-V using FPGA[J].Microprocessors and Microsystems,2023,101

[12] 胡振波.手把手教你设计CPU[M].人民邮电出版社:201806.426.

[13] Cora G ,Sio D C ,Azimi S , et al.Selective hardening of RISCV soft-processors for space applications[J].Microelectronics Reliability,2025,167115667-115667.

[14] Duran C , Wachs M , Luis E R G ,et al.An Energy-Efficient RISC-V RV32IMAC Microcontroller for Periodical-Driven Sensing Applications[J].IEEE, 2020.DOI:10.1109/CICC48029.2020.9075877.

[15] Solodovnikov P A ,Pereverzev L A ,Silantyev M A .Software-Hardware Complex for Accelerating System-on-Chip Design Verification[J].Russian Microelectronics,2025,53(7):722-727.

[16] 唐俊龙,禹智文,刘远治,等.面向RISC-V处理器的GCC移植与优化[J].计算机应用与软件,2021,38(09):262-267+285.

[17] 黄国伟.基于PGL22G的物联网Sensor HUB设计[J].电子产品世界,2019,26(02):45-47.

[18] 唐钏,刘昱,曾林.面向步进电机控制的RISC-V微控制器的设计与实现[J].重庆大学学报,2021,44(11):31-39.

[19] 折如义,李炳辉,姜佩贺.三级流水线RISC-V处理器设计与验证[J].电子技术应用,2020,46(05):44-49.DOI:10.16157/j.issn.0258-7998.200028.
[20] 赵惠洁,杨振荣.基于国产FPGA的数显表设计[J].流体测量与控制,2022,3(01):38-40+49.
[21] 李铭,杨明昕,穆鹏程,等.国产FPGA上通信基带发端算法设计和系统实现[J].实验技术与管理,2024,41(02):135-145.DOI:10.16791/j.cnki.sjg.2024.02.018.
[22] 唐钏.基于RISC-V指令集架构的微控制器设计及应用研究[D].天津大学,2021.DOI:10.27356/d.cnki.gtjdu.2021.002603.
[23] 惠为君.一种SOC片上总线的设计与仿真[J].电脑知识与技术,2021,17(14):206-207.DOI:10.14004/j.cnki.ckt.2021.1308.
[24] 陈龙震,徐康民,徐天骅,等.RISC-V架构的交叉调试系统设计[J].单片机与嵌入式系统应用,2023,23(12):15-18+22.
[25] 黄子晨,李德华.基于OpenOCD和JTAG的嵌入式交叉开发环境[J].仪器仪表用户,2012,19(01):73-75.
[26] 王慧,宋健,王怀斌.嵌入式处理器基准性能测试技术研究[J].航空计算技术,2025,55(01):124-128.
[27] Sahu A ,Mishra S R ,Gour P .Design and Interfacing of High speed model of FPGA using I2C protocol[J].International Journal of Computer Technology and Applications,2011,2(3):531.
[28] 姜贺亮.基于FPGA的32位RISC-V处理器设计[D].兰州大学,2023.DOI:10.27204/d.cnki.glzhu.2023.000436.
[29] 周强.基于RISC-V指令集架构的微控制器的设计与实现[D].电子科技大学,2020.DOI:10.27005/d.cnki.gdzku.2020.003265.
[30] 张龙彪,张果,王剑平,等.嵌入式操作系统FreeRTOS的原理与移植实现[J].信息技术,2012,36(11):31-34.DOI:10.13274/j.cnki.hdzj.2012.11.045.

致　谢

时光荏苒，白驹过隙。转眼间我的本科生活已接近尾声，回首四年的求学生涯，既有收获的喜悦，也有成长的感慨。这篇毕业论文的完成，离不开许多人的关心与帮助。我怀着无比感激的心情，向所有给予我指导与支持的人致以诚挚的感谢。

首先，衷心感谢本科期间每一位悉心教导我的老师们，感谢你们在学术上的严格要求与耐心指导，教会了我如何思考问题、解决问题，更教会了我做人的道理。在人生道路上，您们既是我的良师，更是我的引路人。感谢张喜海老师和张宇老师在毕业论文写作过程中给予的细致指导与耐心帮助，同时在平日的学习交流中也不断为我提供了宝贵的专业建议，帮助我不断提升自身的学习能力和科研素养。同时，我也要感谢宋伟先老师，是他让我初次接触到 FPGA 的世界，激发了我对这一方向的兴趣，并坚定我在研究生阶段继续学习 FPGA 的决心。

其次，感谢我的父母和家人二十余载的养育之恩与无私支持。求学路上，你们始终是我最坚强的后盾，用无条件的爱与包容给予我前行的勇气。愿以今日所学回报你们的期许，未来亦会继续努力，不负所望。感谢我的室友、同学和朋友们，感谢你们在学习和生活中给予的陪伴、鼓励与帮助，让我的大学时光充满了温暖与欢笑。

最后，特别感谢 gitee 平台作者 liangkangnan 提供的 tinyriscv 开源项目参考，为我的毕业设计提供了重要的技术支持和学习借鉴。

纸短情长，寥寥数语难表谢意。愿所有给予我温暖的人平安顺遂，愿自己永葆求知初心，在未来的道路上继续探索前行。再次向所有关心和帮助过我的人表示最真挚的感谢！

学位论文原创性声明

本人郑重声明：所提交的毕业论文是本人在导师指导下独立进行研究工作所取得的成果。对本文的研究工作做出重要贡献的个人和集体，均已在文中以明确方式注明，其它未注明部分不包含他人已发表或撰写过的研究成果，不存在购买、由他人代写、剽窃和伪造数据等学术不端行为。

本人愿为此声明承担法律责任。

作者签名：　　日期：　　年　　月　　日

学位论文版权授权书

本学位论文作者同意授予东北农业大学对本论文全部内容的信息网络传播权、汇编权、发行权、转授权与复制权，具体包括但不限于将学位论文上传到有关数据库，影印、缩印或扫描等手段保存、查阅、借阅、汇编等。

（保密的学位论文在解密后适用本授权书）

作者签名：　　日期：　　年　　月　　日

任务书

学生姓名	睢红鑫	学号	A05210206	班级	物联 2102
题目	基于国产 FPGA 的 RISC-V 软核移植与 FreeRTOS 实现				

毕业论文（设计）要求

1、了解国产 FPGA 与软核处理器的发展现状，按时完成软核控制系统的设计与实现，并完成毕业论文撰写；

2、完成软核处理器的逻辑设计、基础外设控制模块开发及系统集成测试，能够实现软核系统的基本控制与运行功能，并提供完整的设计文档与源代码；

3、按照学校要求提交毕业论文，全文不少于 10000 字，结构合理、内容翔实、图表完整，符合东北农业大学本科毕业论文撰写规范。

毕业论文（设计）内容与技术参数

内容：

1、阐述国产 FPGA 平台软核控制设计的研究目的与意义，分析在嵌入式系统中的应用价值；

2、设计并实现一款基于 RISC-V 指令集的软核处理器，包括指令译码、执行单元和寄存器组等基本功能模块；

3、完成软核平台的外设控制功能开发，包括 GPIO、UART、Timer 等模块。

4、构建软核系统综合运行平台，实现简单任务调度机制，具备一定的可扩展性和演示能力。

技术参数：

开发语言：Verilog HDL、C 语言

开发工具：紫光 FPGA、 PDS、串口调试助手、ModelSim、GNU 工具链

毕业论文（设计）工作计划

2024.12-2025.01　查阅国产 FPGA 和软核处理器相关发展资料，了解软核在嵌入式系统中的应用，明确设计目标和需求分析。

2025.01-2025.02　完成软核处理器基本架构设计，包括指令集选择、模块划分与初步仿真。

2025.02-2025.03　完成软核核心模块开发与调试，实现 UART、GPIO 等外设功能；初步构建系统平台，进行联调测试。

2025.03-2025.05　进一步完善软硬件功能，开展系统演示，撰写毕业论文，准备答辩。

接受任务日期：2024 年 12 月 01 日　**要求完成日期：**2025 年 05 月 08 日

学　　生：睢红鑫

指 导 教 师：张喜海

东北农业大学毕业论文（设计）开题报告

学生姓名	睢红鑫	学号	A05210206	班级	物联 2102
题目	基于国产 FPGA 的 RISC-V 软核移植与 FreeRTOS 实现				

题目研究的目的和意义

21 世纪以来，信息技术快速演进，国家对集成电路产业的自主发展高度重视。尤其在“新基建”“国产化替代”等战略背景下，芯片自主可控成为国家科技安全的重要一环。作为重要的可重构硬件平台，FPGA（现场可编程门阵列）以其高灵活性、并行处理能力和硬件级可定制性，越来越广泛地应用于工业控制、网络通信、图像处理、人工智能加速等关键领域。特别是近年来国产 FPGA 的快速发展，打破了对国外技术和供应链的依赖，为我国在关键行业和领域中构建自主、可靠的计算平台提供了基础保障。

软核处理器作为一种由硬件描述语言实现的处理器架构，运行于 FPGA 中，相较于硬核处理器具有更强的灵活性、可裁剪性与可扩展性。在应用中，开发者可以根据系统需求灵活修改软核结构、扩展指令集或集成专用加速器，从而实现资源最大化利用和系统定制化开发。同时，在安全性和自主可控方面，软核处理器的可见性和可修改性也使其在军事、工业等领域有着不可替代的价值。然而，当前国内对于软核处理器的研究与实践尚处于发展阶段，尤其是在国产 FPGA 平台上的应用与优化仍存在较大探索空间。本设计旨在构建一个具备自主可控能力的软核处理平台，基于国产 FPGA 芯片，验证其在关键领域中实际可行性和应用潜力。

设计的主要目标是基于紫光同创盘古系列国产 FPGA 平台，完成一款 RISC-V 指令集架构的简洁、实用的软核处理器设计，集成包括 GPIO、UART、定时器等基础外设模块，并搭建基础的任务调度。在此基础上，完成基本软件开发与硬件验证流程，实现可运行的系统控制平台，具备一定的可拓展性与移植能力。研究过程中，将深入涉及指令集架构设计、流水线控制逻辑实现、片上通信总线协议、硬件外设控制逻辑等方面的内容，进一步完成软硬件接口设计、底层驱动开发与调试工作，形成一个软核系统开发的完整流程。

从更高层次来看，软核处理器与国产 FPGA 平台的深度结合，是实现信息系统安全可控、推动自主嵌入式计算平台构建的重要路径。目前市面上的主流软核如 MicroBlaze、Nios II 等仍依赖于国外厂商，存在架构封闭、源码不公开等问题，难以满足当前高安全性和深度定制的应用需求。本设计采用开源、开放的 RISC-V 架构，并在国产 FPGA 平台上实现与验证，具备良好的可控性、可移植性和灵活性，有助于构建自主知识产权的嵌入式计算平台。此外，本项目设计的软核平台具有良好的可裁剪性与模块化结构，可根据特定应用场景灵活集成加密模块、通信接口协议或定制化加速器，在边缘计算、工业自动化、信息安全等领域具备较好的适应性和推广潜力。项目完成后的系统不仅可作为基础控制平台进一步扩展，还可为后续研究如多核系统、SoC 设计、软硬件协同优化等提供技术支撑。本设计聚焦于国产化 FPGA 与 RISC-V 软核平台的融合，具有重要的工程实践意义和自主可控研究价值。通过对软核控制平台的设计与调试，不仅可以推进相关技术在实际中的落地

与应用，也将为今后开展更复杂嵌入式系统研发工作打下坚实的基础。

题目研究方案和主要内容

本设计首先调研了国内外软核处理器、国产 FPGA 及嵌入式系统平台的发展现状，分析了当前在国产平台上进行软核处理器开发与系统应用面临的挑战与机遇。在此基础上，明确了构建自主可控软核平台并实现基础控制功能的必要性，提出了基于国产 FPGA 的软核控制系统整体设计方案。接着本研究基于国产 FPGA 平台——紫光同创盘古系列开发板，设计并实现一个基于 RISC-V 指令集架构的软核处理器。处理器将集成基础外设控制接口模块，包括 GPIO、UART 串口、定时器等，满足基础 IO 控制、串口通信和定时中断等功能需求。软核整体采用 Verilog 语言进行描述与调试，并通过 FPGA 平台进行功能验证。在平台验证过程中，设计自定义测试程序以运行在软核上，检验指令执行正确性和外设读写的稳定性。为进一步提升系统的可拓展性，本设计基于 FreeRTOS 实时内核进行了移植适配，构建了简单的多任务运行环境。

研究方法及技术途径

本设计采用软硬件协同设计方法，基于国产 FPGA 平台，致力于设计并实现一个简洁高效的软核处理器。首先，利用 Verilog 语言实现 RISC-V 指令集架构的五级流水线软核处理器，设计并调试核心模块，如指令取出、译码、执行、访存和写回模块，确保指令的正确执行。同时，结合 FPGA 平台的特性，设计寄存器堆、运算单元和内存接口等功能模块，完成软核处理器的基本框架。接下来，研究并设计系统的外设接口，包括 GPIO、UART、定时器等常用模块，为后续任务调度和资源管理提供硬件支持。外设接口设计时，考虑到可扩展性和模块化，采用总线结构实现不同外设的集成，确保软核处理器能高效地与外部设备进行通信和控制。此外，设计中断控制器模块，支持硬件中断的响应和外设的管理，使系统能够高效处理多任务场景。在软核处理器的硬件设计完成后，本设计将移植一个轻量级实时操作系统（如 FreeRTOS）到软核平台。

实施计划

2024.12-2025.01　查阅国产 FPGA 和软核处理器相关发展资料，了解软核在嵌入式系统中的应用，明确设计目标和需求分析。

2025.01-2025.02　完成软核基本架构设计，包括指令集选择、模块划分与初步仿真。

2025.02-2025.03　完成软核核心模块开发与调试，实现 UART、GPIO 等外设功能；初步构建系统平台，进行联调测试。

2025.03-2025.05　完善软硬件功能，开展系统演示，撰写毕业论文，准备答辩。

指导教师：张喜海

审核时间：2025 年 04 月 08 日

东北农业大学毕业论文（设计）中期检查

学生姓名	睢红鑫	学号	A05210206	班级	物联 2102
题目	基于国产 FPGA 的 RISC-V 软核移植与 FreeRTOS 实现				

论文概述

论文由六章组成，章节组织结构如下：

第一章为绪论，介绍课题背景、研究意义、国内外研究现状以及本论文的研究目标与内容安排，明确论文的研究方向和实际应用价值。

第二章为微控制器相关的研究理论与技术实现，主要介绍软核处理器设计所需的基础理论知识，包括 RISC-V 指令集架构、软硬件协同设计原理及 Verilog 语言实现方法

第三章为 RISC-V 微控制器的设计与实现，详细阐述软核处理器的总体架构设计及核心功能模块（如控制单元、寄存器组、运算单元等）的开发过程，是论文的核心部分之一。

第四章微控制器的外围设备设计与 FPGA 验证，介绍包括 GPIO、UART、Timer 等外设模块的设计方法及其与软核系统的集成，并通过 FPGA 平台完成上板测试与功能验证。

第五章基于 RISC-V 的跑分验证与 RTOS 移植，在完成软核处理器设计后，通过运行性能测试程序对其功能进行评估，并完成 FreeRTOS 的移植与基础任务调度功能的实现。

第六章总结与展望，总结全文研究成果，分析软核系统设计中的不足之处，并对未来国产 FPGA 平台上软核系统的优化与应用提出展望方向。

本阶段研究工作的内容和进展

1、查阅了近年来国产 FPGA 的发展情况，了解了国内外关于 RISC-V 软核处理器及嵌入式操作系统移植的研究动态，并完成了项目的需求分析与技术选型；

2、通过知网等专业数据库，深入阅读了大量关于 RISC-V 架构设计、软硬件协同开发及 FPGA 验证平台的文献资料，对比分析了各类软核设计方案与实际应用路径，为本课题提供理论支持；

3、围绕 RISC-V 架构及国产 FPGA 平台（如紫光盘古系列），开展了处理器结构、流水线控制、指令集实现、总线接口等核心技术的学习与研究，掌握了软核构建的基本流程和关键技术；

4、确定了基于 Verilog 的软核实现路径，完成了具有基本指令集支持的 RISC-V 处理器的搭建工作，并成功在国产 FPGA 平台上实现部署和仿真；

5、在外围接口方面，已完成 GPIO、UART 等基础外设的设计与调试，验证了软核与外设之间的数据通信能力，为后续操作系统调度和中断机制奠定基础；

6、初步完成了 FreeRTOS 移植所需的软硬件适配准备工作，包括中断响应机制、任务调度框架分析等内容，并开始编写用于系统调用与任务管理的接口代码。

遇到的问题

1、在操作系统移植过程中，面临较大的裁剪和适配工作。由于 RISC-V 软核处理器指令集较为简洁，无法完整支持部分操作系统默认依赖的扩展指令或特性，因此需要对 FreeRTOS 进行裁剪和配置调整，剔除不兼容模块，并手动编写与平台相关的端口代码（如中断管理、任务切换等），这一过程工作量较大，调试难度高。

2、在软核系统整体性测试阶段也面临不少困难。当前处理器功能模块虽已基本实现，但由于各个模块（如流水线、总线、外设）间存在一定耦合关系，导致系统级联调中出现多个不易定位的问题，如中断响应异常、数据传输不一致、寄存器状态异常等。这些问题暴露出软核设计在系统集成层面的不足，仍需反复调试和优化。

下一步工作

1、继续完善软核处理器设计，对现有模块进行优化，进一步细化控制逻辑和指令执行流程，提升整体稳定性和执行效率，增强与外设的协同能力，为系统级移植提供可靠基础。

2、补全和调试各类外设接口模块，包括但不限于 UART、GPIO、Timer 等常用外设，并在 FPGA 平台上进行功能验证，确保外设可与处理器顺利通信，支撑后续操作系统运行需求。

3、完成 FreeRTOS 在 RISC-V 软核上的裁剪与完整移植，重点实现任务调度、中断管理、系统节拍等核心功能，并结合软核结构进行适配和优化，确保其在硬件平台上能稳定运行。

4、开展软硬件协同测试与调试，包括处理器、外设、操作系统等模块的集成验证，及时排查功能不一致、信号冲突等问题，提升系统整体可靠性。

5、加强与指导教师的沟通交流，对设计方案与实现路径进行持续评估，根据项目实际进展与问题反馈，及时调整开发节奏与技术路线。

6、完善文档与实验记录，为后续论文撰写和答辩材料准备打下基础，同时积累开发过程中关键技术细节与调试经验，以便复用和推广。

指导教师：张喜海

审核时间：2025 年 04 月 08 日

指导教师评语

学生姓名	睢红鑫	学号	A05210206	班级	物联 2102
题目	基于国产 FPGA 的 RISC-V 软核移植与 FreeRTOS 实现				

指导教师评语：

该论文基于国产紫光同创 PGL22G FPGA 完成了 tinyriscv RISC- V 软核的移植与 FreeRTOS 系统整合，实现了软硬件协同验证及性能评估，构建了完整的国产化嵌入式原型系统。作者对 RISC- V 软核体系结构与 FPGA 资源进行了深入分析，能够独立完成处理器流水线、总线接口、BRAM IP 适配及外设设计，方法规范、实验充分；在软件层面成功移植交叉编译链与 FreeRTOS，并借助 CoreMark、OpenOCD 等工具进行功能与性能测试，论证过程严谨，结果可信；论文结构清晰、论述条理，资料引用恰当，展现了较强的工程实践与科研创新能力，具有一定的实用价值。

论文查重和 AIGC 检测结果均符合学校要求，同意参加答辩。

指导教师：张喜海

评阅时间：2025 年 05 月 04 日

评阅人意见

学生姓名	睢红鑫	学号	A05210206	班级	物联 2102
题目	基于国产 FPGA 的 RISC-V 软核移植与 FreeRTOS 实现				

评阅人意见：

论文围绕物联网开发中国产化嵌入式系统开发需求，基于紫光同创 PGL22G FPGA 平台，成功实现了 tinyriscv RISC-V 软核的移植与 FreeRTOS 实时操作系统的整合工作。在硬件设计方面，作者系统性地完成了 RISC-V 处理器流水线优化、AXI 总线接口适配、BRAM 存储控制器定制以及外设驱动开发等关键任务，充分展现了扎实的硬件设计能力；在软件层面，不仅成功构建了完整的 RISC-V 交叉编译工具链，还实现了 FreeRTOS 任务调度器与底层硬件的无缝对接。研究采用 CoreMark 性能基准测试与 OpenOCD 调试工具对系统进行了全面验证，测试数据详实可靠。论文工作技术路线清晰合理，实验设计科学规范，文档资料完整准确，研究成果对推动国产芯片在嵌入式领域的应用具有实际参考价值，体现了作者在嵌入式系统开发方面的专业素养和创新能力。

、

评阅人：张宇

评阅时间：2025 年 05 月 05 日

答辩委员会意见

<table>
<tr><td>学生姓名</td><td>睢红鑫</td><td>学号</td><td>A05210206</td><td>班级</td><td>物联 2102</td></tr>
<tr><td>题目</td><td colspan="5">基于国产 FPGA 的 RISC-V 软核移植与 FreeRTOS 实现</td></tr>
<tr><td colspan="6">答辩委员会意见：
毕业论文全面完成了毕业设计任务书规定的内容、要求；能灵活运用所学的理论知识解决设计中的问题，有独到见解；
论文设计合理，计算结果正确，图表清晰，表达准确，论文的系统性、逻辑性较强，行文流畅，文字简练；答辩时回答问题思路清晰、正确。经答辩委员会讨论，一致同意该生通过论文答辩，并建议授予工学学士学位。

答辩委员会主席：冯江</td></tr>
<tr><td>等级</td><td>优秀</td><td colspan="4">院长：李晓明

2025 年 05 月 30 日</td></tr>
</table>

电气与信息学院本科毕业论文（设计）评价指标

指标	评价要素		评价指标内涵
选题质量	1	目的明确，符合要求	符合培养目标，体现学科、专业特点和教学计划中对能力、知识结构的基本要求，达到毕业综合训练的目的。
	2	理论意义或实际价值	符合本学科的理论发展，解决学科建设、科学发展的理论或方法问题，有一定的科学意义，符合我国经济建设和社会发展的需要，解决应用性研究中的某个理论或方法问题，具有一定的实际价值。
	3	选题适当	题目贴切，有较强的科学性，难易度适中，题目规模适当。
能力水平	4	查阅文献资料能力	能独立查阅相关文献资料，归纳总结本领域有关科学成果。
	5	综合运用知识能力	能运用所学专业知识分析论述有关问题，能对占有资料进行分析整理并适当运用，概念清楚，能以恰当的论据对科学论点进行有说服力的论证。
	6	研究方案的设计能力	设计（论文）的整体思路清晰，结构完整，研究方案完整有序。
	7	研究方法和手段的运用能力	能较熟练的运用本学科的常规科学研究方法，能适当运用相关研究手段（如计算机、实验仪器设备等）进行资料搜集、加工、处理，并辅助论文写作。
	8	外文应用能力	能按照学校规定结合论文阅读、翻译一定量的本专业外文资料，有外文摘要和外文书目。
成果质量管理	9	文题相符	论文较完整地回答了题目所设定的有关问题。
	10	写作水平	论点鲜明，观点正确，论据充分，论证有力，条理分明，语言流畅。
	11	写作规范	符合科学论文的基本要求，论文（设计）中的用语格式、图表、数据、各种资料的运用及引用都要规范化。
	12	篇幅	工程设计类篇幅不少于 8000 字，研究论文篇幅不少于 10000 字。
	13	成果的理论或实际价值	在理论上具有新意，应用性研究对于实际工作具有一定意义。